Dossier projet
Concepteur Développeur d'Applications (CDA)

Année 2024 – 2025



Find, Rate, Collect ....

Matthieu Nowak

Encadrants

Mme Laury BOSSAERT — Responsable de formation
M.Jimmy JAWOROWSKI — Tuteur entreprise (Almadia)

Date de rendu : 30 Juin 2025





Remerciements

Avant d'entrer dans le vif du sujet de ce projet, il me tient à cœur de commencer par exprimer ma gratitude envers celles et ceux qui ont contribué à cette expérience.

Je tiens tout d'abord à remercier chaleureusement M. Romain Watremez, directeur général d'Almadia, pour m'avoir accueilli au sein de son entreprise.

Ma reconnaissance s'adresse également à M. Jimmy Jaworowski, directeur informatique et tuteur en entreprise, pour sa confiance, son accompagnement, ainsi que pour le partage de son expertise et sa grande disponibilité.

Je remercie également l'ensemble de l'équipe informatique, qui m'a non seulement intégré avec bienveillance, mais aussi transmis ses compétences. Au-delà de la collaboration professionnelle, une véritable complicité s'est installée, et j'ai eu la chance de nouer des liens solides en dehors du cadre du travail.

Enfin, je souhaite exprimer toute ma reconnaissance à ForEach Academy et à l'équipe pédagogique, notamment Mme Laury Bossaert, pour m'avoir accueilli dans le cursus Concepteur Développeur d'Applications (CDA) et accompagné dans ma progression, tant sur le plan technique que personnel.

Sommaire

1-Introduction et Cahier des charges.....5
1.1-Qui suis-je ?.....5
1.2-Présentation du projet.....5
1.3-A qui s'adresse cette application ?.....6
1.4-Contexte du marché.....6
1.5-Analyse des besoins.....7
1.6-Enjeux et contraintes.....7
1.7-Solutions.....8
1.8-Architecture du projet.....9
1.9-Parcours Utilisateur.....10
1.10-User Story et test d'acceptances.....11
User Story : Recherche de jeu et ajout à une collection personnalisé.....11
Titre :.....11
Description :.....11
Critères d'acceptation :.....11
Tests d'acceptation.....11
Scénario 1 : Recherche de jeu réussie et ajout à la collection.....11
Scénario 2 : Aucune correspondance trouvée.....12
Scénario 3 : Utilisateur non connecté.....12
1.11-Liste des compétences du référentiel.....13
2-Maquette de l'application.....13
3-Mise en place du projet.....14
3.1-Gestion du projet.....14
3.2-Installation du projet.....14
4-API IGDB.....16
4.1-Présentation de l'API.....16
Intégration dans Gamelab.....16
4.2-Obtention des accès.....17
4.3-Test de l'API.....18
5-API Gamelab.....19
5.1-Technologies utilisées.....19
5.2-Fonctionnalités principales de l'API.....19
5.3-Base données de l'application.....20

5.4-Gestion des utilisateurs....21
5.4.1-Inscription....21
5.5-Fonctionnalités Principales....25
5.5.1-Ajout dans une collection (Favoris)....25
5.5.2-Création d'une collection personnalisée....25
5.5.3-Notation d'un jeu....26
5.5.4-Système d'aministration (Back-office)....26
5.6-Test Unitaire....28
5.6.1-Création d'un utilisateur et connexion....28
5.6.3-Ajout d'un jeu en favoris....30
5.6.3-Test non fonctionnel....31
5.7 – Dockerisation de l'API....32
6-Landing Page....34
6.1-Maquette....34
6.2-Design basé sur l'aléatoire....34
7-Application Web....35
7.1-Récupération des différentes API et utilisation....35
7.1.1 Service de l'api Gamelab....36
7.1.1 Service de l'api IGDB....37
7.2-Back-office....39
7.3-Système d'ajout dans une collection....40
8-Application Mobile....42
8.1-Pourquoi avoir choisi une application mobile séparée ?....42
8.2-Récupération des différentes API et utilisation....43
8.2.1 Services de l'api Gamelab....43
8.2.2 Services de l'api IGDB....45
8.3-Création de l'APK et installation de cette dernière....46
9-Futures Améliorations du projet....47
10-Conclusion....48
11-Annexes....49

1-Introduction et Cahier des charges

1.1-Qui suis-je ?

Avant d'introduire mon projet, permettez-moi de me présenter. Je m'appelle Matthieu Nowak, j'ai 23 ans et je suis passionné de jeux vidéo depuis mon plus jeune âge. Cette passion m'a poussé à créer mes premiers jeux sur des plateformes comme GameMaker Studio et Roblox, ce qui m'a naturellement ouvert les portes du développement informatique. Au fil du temps, je me suis découvert un véritable intérêt pour le développement web et mobile, domaine dans lequel je m'épanouis aujourd'hui.

Après avoir suivi une formation de Développeur Web et Web Mobile (DWWM) chez Pop School à Valenciennes, j'ai décidé de poursuivre mon parcours avec la formation de Concepteur Développeur d'Applications (CDA) en alternance, proposée par ForEach Academy. Mon alternance se déroule chez Almadia, une entreprise accès dans le médical et le maintien à domicile, où je travaille principalement sur des projets développés avec le framework Symfony.

1.2-Présentation du projet

Gamelab est une application multiplateforme accessible à la fois sur le web et sur mobile. Elle permet aux utilisateurs de gérer facilement leurs collections de jeux vidéo, de créer des listes personnalisées selon leurs envies, et de découvrir de nouveaux jeux à essayer. Grâce à son système de recherche à filtres, trouver un jeu devient rapide et intuitif.

L'application intègre également une dimension communautaire : les utilisateurs peuvent noter les jeux et laisser des avis. Que ce soit pour suivre les tendances ou partager ses coups de cœur, Gamelab offre une expérience riche et personnalisée.

Elle s'appuie sur l'API IGDB (Internet Game Database), qui permet d'accéder à une base de données de plus de 400 000 jeux (dont DLC et Remastered) couvrant toutes les plateformes. Gamelab vise à devenir un outil central pour tous les joueurs de jeux vidéo.

Source : https://en.wikipedia.org/wiki/IGDB

> As of March 2023, IGDB has 99,000 members and encompasses 428,000 games (of which 196,500 are DLCs and re-releases), 44,300 companies and 217,000 people as well as over 1.1 million reviews, screenshots and videos.[*citation needed*]

1.3-A qui s’adresse cette application ?

Gamelab s’adresse à toutes les personnes qui aiment les jeux vidéo, qu'ils soient collectionneurs, joueurs occasionnels ou joueurs réguliers. L’application est conçue pour être utile quel que soit le nombre de plateformes que vous utilisez. Que vous possédiez des jeux uniquement sur une seule plateforme ou sur plusieurs (PC, consoles, mobile), Gamelab vous offre une solution efficace pour organiser et gérer votre collection.

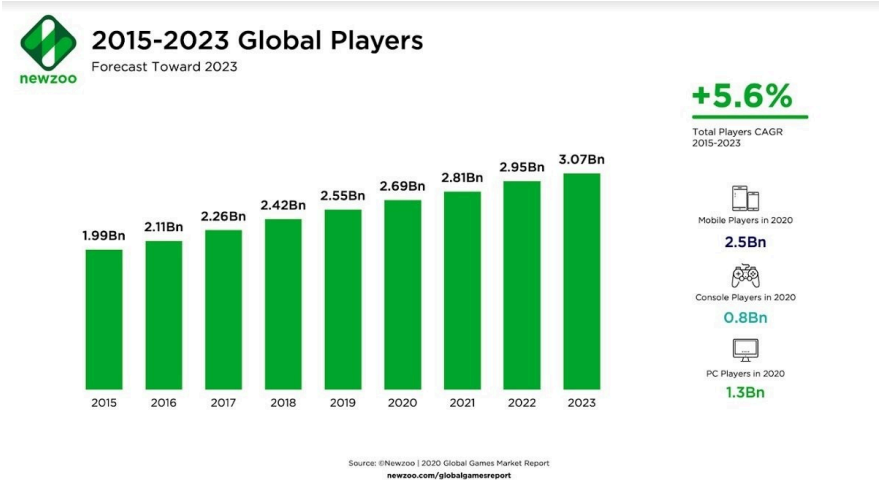
Elle est également idéale pour les joueurs souhaitant découvrir de nouveaux titres, consulter les avis de la communauté, ou encore noter et analyser leurs expériences de jeu.

Enfin, elle s’adresse aux créateurs de contenu, critiques de jeux ou influenceurs qui souhaitent partager leurs recommandations.

1.4-Contexte du marché

En 2023, l'industrie des jeux vidéo a connu un dynamisme sans précédent avec le lancement de plus de 14 500 nouveaux titres, soit environ 40 jeux par jour en moyenne. Certains jours ont même enregistré jusqu'à 50 ou 60 nouvelles sorties simultanées, engendrant une saturation du marché qui rend la visibilité et la notoriété des jeux particulièrement difficiles à atteindre.

Le nombre de joueurs continue de croître de manière exponentielle, atteignant plus de 4 milliards dans le monde en 2024. Cette immense base de joueurs est accompagnée par un phénomène financier notable : environ 19 milliards de dollars de jeux restent non joués sur Steam en 2024. Cela révèle un potentiel inexploité considérable et souligne la nécessité d'outils facilitant la découverte et l'organisation des jeux dans cet environnement saturé.



Source: https://www.journaldugeek.com/2024/06/28/steam-les-joueurs-depensent-des-milliards-pour-des-jeux-auxquels-ils-nont-jamais-touche/

1.5-Analyse des besoins

Avec plus de 400 000 titres disponibles, les défis liés à la gestion et à la découverte des jeux deviennent de plus en plus complexes pour les joueurs. Les besoins comprennent une :

- Découverte améliorée : Les joueurs cherchent des moyens plus efficaces pour découvrir de nouveaux jeux.
- Organisation personnalisée : Les utilisateurs ont besoin d'outils pour organiser leur collection de jeux, suivre leur progression, et gérer les jeux joués et non joués.
- Interaction sociale : Les joueurs veulent interagir avec une communauté active pour partager des recommandations et des expériences.

Gamelab doit répondre à ces besoins en offrant une plateforme qui facilite la découverte, l'organisation et l'interaction autour des jeux vidéo.

1.6-Enjeux et contraintes

Avant de parler des fonctionnalités de Gamelab, je pense que c'est important de poser le contexte. Le projet doit faire face à plusieurs enjeux importants, mais aussi à quelques contraintes techniques et matérielles qu'il ne faut pas négliger.

Enjeux :

- Saturation du marché : Le secteur des applications de gestion de jeux est déjà bien rempli. Gamelab doit donc se distinguer par des fonctionnalités originales et pertinentes afin d'attirer et de fidéliser les utilisateurs.
- Adoption multiplateforme : L'application vise une disponibilité sur plusieurs supports (Android, web). Il est donc essentiel d'assurer une expérience utilisateur fluide et cohérente, quel que soit l'environnement.
- Croissance des utilisateurs : Le public visé est large et en croissance. Pour rester pertinent, Gamelab devra évoluer régulièrement grâce à des mises à jour adaptées aux attentes des joueurs.

Contraintes :

- Absence sur iOS : Faute de matériel Apple, l'application n'est pas encore disponible sur iOS. Cette limitation réduit l'audience potentielle, mais pourrait être levée avec un financement dédié.
- Ressources financières et techniques : Le développement et la maintenance d'une application multiplateforme demandent des moyens techniques et financiers importants. Il est donc nécessaire de bien planifier les ressources pour garantir un service fiable et évolutif."
- Application en Anglais : L'application, reprenant l'API IGDB n'est pas multilingue et n'est disponible qu'en anglais.

Ces éléments doivent être pris en compte tout au long du développement afin d'assurer la viabilité du projet, d'anticiper les risques et de maximiser les chances de succès de Gamelab sur un marché exigeant et en constante évolution.

1.7-Solutions

Pour faire face aux enjeux identifiés et répondre efficacement aux contraintes techniques et financières, plusieurs solutions stratégiques sont envisagées afin d'assurer le bon développement, la viabilité et la croissance de Gamelab.

Financement participatif :

- Lancement d'une campagne de crowdfunding : Le recours à une campagne de financement participatif pourrait être une solution pertinente pour faciliter l'intégration future de Gamelab sur iOS, enrichir les fonctionnalités de la plateforme et assurer sa maintenance dans la durée. Cette option permettrait également d'impliquer la communauté dès les premières étapes du projet et de fédérer un groupe d'utilisateurs engagés.

Partenariats stratégiques :

- Collaboration avec des influenceurs : La mise en place de partenariats avec des influenceurs spécialisés dans le domaine du jeu vidéo est également à l'étude. Ce levier permettrait de promouvoir l'application auprès d'un public ciblé, tout en bénéficiant de recommandations crédibles.
- Recherche de partenaires d'acquisition : Une des idées serait aussi de trouver un partenaire d'acquisition qui aimerait acquérir des parts ou l'entièreté de l'application à long terme.

Publicité et promotions :

- Publicités ciblées : Intégrer des publicités ciblées pour générer des revenus complémentaires pourrait-être envisagé. Cela dit, cette solution reste secondaire pour le moment, l'objectif étant avant tout de privilégier les options de financement direct et les partenariats.
- Promotions et offres spéciales : Des campagnes promotionnelles et des offres spéciales pourraient être mises en place pour attirer de nouveaux utilisateurs et encourager les abonnements premium ou les mises à niveau.

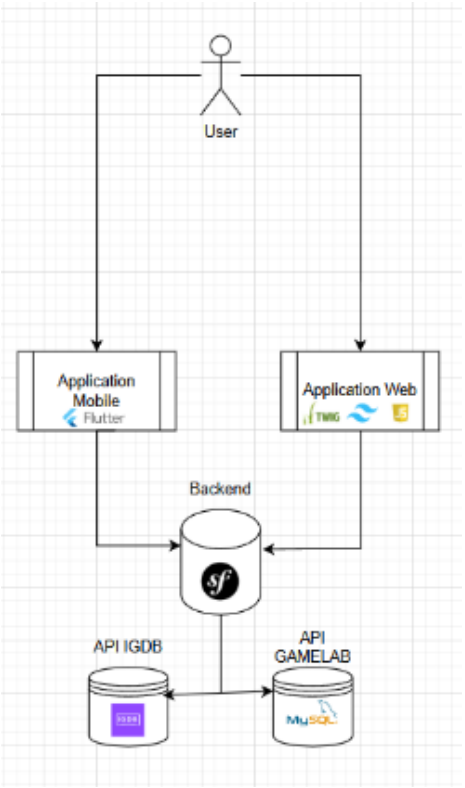
Achats intégrés :

- Fonctionnalités premium : L'ajout de fonctionnalités premium via des achats intégrés est une option envisagée, mais ce n'est pas une direction que je souhaite privilégier. Ce modèle ne serait envisagé qu'en dernier recours, si aucune autre solution ne permettait d'assurer la viabilité du projet. L'objectif est de préserver une expérience utilisateur fluide et accessible à tous, sans créer de barrières ou de frustrations liées à des contenus bloqués.

Ces solutions sont conçues pour surmonter les défis financiers et opérationnels tout en maximisant les opportunités de croissance et de visibilité pour Gamelab. Les options telles que la publicité et les achats intégrés sont envisagées avec prudence, en mettant l'accent sur l'amélioration de l'expérience utilisateur et la création de valeur avant leur intégration.

1.8-Architecture du projet

Après avoir présenté le projet en définissant sa nature, son public cible ainsi que les enjeux et contraintes associés, je souhaite désormais en détailler l'architecture à l'aide d'un schéma explicatif.



Le schéma représente l'architecture générale du projet, articulée autour d'un backend central et de deux interfaces principales accessibles par les utilisateurs.
L'utilisateur peut interagir avec le système via deux types d'applications :

- Une application mobile, développée avec Flutter, qui permet une compatibilité multiplateforme (Android et IOS (la version IOS n'est pas encore disponible comme vu précédemment mais reste une envie de ma part de l'intégrer)) et une interface utilisateur réactive.

- Une application web, conçue à l'aide des technologies Twig et HTML5, offrant une expérience fluide depuis un navigateur classique.

Ces deux interfaces communiquent avec un backend développé en Symfony, qui joue un rôle central dans la gestion de la logique métier. Il traite les requêtes des utilisateurs, orchestre les échanges de données et assure la cohérence de l'ensemble du système.

Le backend interagit avec deux sources de données externes via des APIs :

- L'API IGDB (Internet Games Database), qui permet d'accéder à une base de données complète de jeux vidéo, incluant des informations telles que les titres, genres, dates de sortie, studios, plateformes, etc.

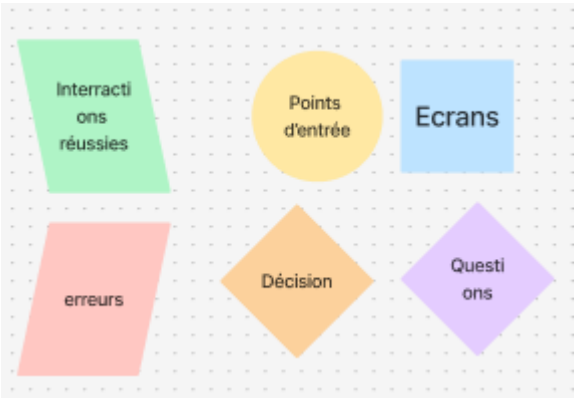
- L'API Gamelab, connectée à une base de données SQL (via MariaDb), qui centralise les données spécifiques au projet, qu'elles soient générées en interne ou issues de traitements métier.

Cette architecture permet une séparation claire des responsabilités entre l'interface utilisateur, la logique applicative et les sources de données.

1.9-Parcours Utilisateur

Afin de pouvoir concevoir une expérience utilisateur cohérente, fluide et centrée sur les besoins réels de l'utilisateur, il est essentiel d'établir un parcours utilisateur (user flow). Ce parcours représente visuellement les étapes clés que suit un utilisateur pour accomplir une tâche spécifique au sein de notre application ou jeu. (Annexe 1)

Voici la légende de mon userflow :



1.10-User Story et test d'acceptances

De plus, pour pouvoir garantir que le produit livré répond bien aux besoins fonctionnels et aux exigences définies en amont, il est essentiel de réaliser des user stories tests d'acceptance (ou tests d'acceptation utilisateur).

J'ai décidé d'en faire un concernant la rechercher d'un jeu et l'ajout à une collection, il a été réalisé avant de commencer le code :

User Story : Recherche de jeu et ajout à une collection personnalisé

Titre :

En tant qu'utilisateur, je veux pouvoir rechercher un jeu et l'ajouter à ma collection pour organiser mes jeux préférés.

Description :

En tant qu'utilisateur de Gamelab, je souhaite rechercher un jeu dans la base de données via la barre de recherche afin de pouvoir l'ajouter facilement à mes collections organisées dans des dossiers.

Critères d'acceptation :

- L'utilisateur peut entrer le nom du jeu dans la barre de recherche.
- Une liste de résultats pertinents s'affiche en temps réel pendant la saisie.
- L'utilisateur peut sélectionner un jeu parmi les résultats affichés.
- Après sélection, l'utilisateur a l'option d'ajouter le jeu à une collection existante ou d'en créer une nouvelle.
- Le jeu est visible dans la collection après ajout.
- Un message de confirmation s'affiche une fois le jeu ajouté avec succès.

Tests d'acceptation

Scénario 1 : Recherche de jeu réussie et ajout à la collection

- **Étant donné** qu'un utilisateur est connecté à Gamelab,
- **Quand** il tape "Zelda" dans la barre de recherche,
- **Alors** une liste de résultats correspondant aux jeux "Zelda" s'affiche.
- **Quand** il sélectionne "The Legend of Zelda: Breath of the Wild" dans les résultats,
- **Et** qu'il choisit l'option "Ajouter à la collection",
- **Et** qu'il choisit un dossier existant ou créer un nouveau dossier nommé "Zelda",
- **Alors** le jeu est ajouté à la collection "Zelda" ou au dossier existant,
- **Et** un message de confirmation s'affiche : "The Legend of Zelda: Breath of the Wild ajouté avec succès à Zelda."

Scénario 2 : Aucune correspondance trouvée

- **Étant donné** qu'un utilisateur est connecté à Gamelab,
- **Quand** il tape "Titre de Jeu inexistant" dans la barre de recherche,
- **Alors** un message s'affiche : "Aucun jeu correspondant à votre recherche."

Scénario 3 : Utilisateur non connecté

- **Étant donné** qu'un utilisateur n'est pas connecté à Gamelab,
- **Quand** il consulte une page de jeu (par exemple, "The Legend of Zelda: Breath of the Wild"),
- **Alors** il peut voir les détails du jeu.
- **Quand** il essaie d'ajouter le jeu à sa collection,
- **Alors** un message d'erreur s'affiche : "Erreur : veuillez vous connecter."

Ce user story est viable aussi pour les collections déjà existantes comme les collections “joué , favoris, wishlist et complété”.

1.11-Liste des compétences du référentiel

Vue synoptique de l'emploi-type

N° Fiche AT	Activités types	N° Fiche CP	Compétences professionnelles
1	Développer une application sécurisée	1	Installer et configurer son environnement de travail en fonction du projet
		2	Développer des interfaces utilisateur
		3	Développer des composants métier
		4	Contribuer à la gestion d'un projet informatique
2	Concevoir et développer une application sécurisée organisée en couches	5	Analyser les besoins et maquetter une application
		6	Définir l'architecture logicielle d'une application
		7	Concevoir et mettre en place une base de données relationnelle
		8	Développer des composants d'accès aux données SQL et NoSQL
3	Préparer le déploiement d'une application sécurisée	9	Préparer et exécuter les plans de tests d'une application
		10	Préparer et documenter le déploiement d'une application
		11	Contribuer à la mise en production dans une démarche DevOps

2-Maquette de l’application

Je me suis inspiré du site Backloggd ainsi que de la page de bibliothèque Steam pour concevoir l’interface de mon site.

Pour la réalisation des maquettes, j’ai utilisé Figma. J’ai commencé par créer le logo, en testant plusieurs versions afin d’explorer différentes approches visuelles.

Version 1



Version 2 (finale)



Ensuite, j’ai élaboré les maquettes en adoptant une approche mobile first : je me suis d’abord concentré sur l’affichage sur smartphone (Annexe 2), puis j’ai adapté ces maquettes aux formats desktop (Annexe 3).

Le vert utilisé dans l’identité graphique est directement inspiré du “lab” de Gamelab, en référence au mot “laboratoire”. Quant au mot “game”, il signifie “jeu” en anglais. J’ai choisi ce nom pour évoquer un espace dédié aux jeux vidéo, un véritable laboratoire numérique où l’on peut explorer, partager et découvrir de nombreuses fonctionnalités autour de l’univers vidéoludique.

3-Mise en place du projet

3.1-Gestion du projet

Pour organiser efficacement le suivi du projet, plusieurs outils ont été utilisés en fonction des besoins :

- Trello : Mis en place pour la gestion des bugs, des retours et des propositions d’amélioration. Un tableau structuré permettait de suivre l’évolution de chaque problème identifié, les solutions envisagées et les actions mises en œuvre. (Annexe 4)
- Git : Le versionnement du code a été assuré via Gitlab, avec une bonne pratique de gestion des branches. J’ai également utilisé Gitmoji, un système d’émojis pour rendre les messages de commit plus lisibles et expressifs, facilitant ainsi le suivi des modifications. (Annexe 5)

3.2-Installation du projet

Le projet se compose de trois applications développées avec Symfony : la landing page, la version web de Gamelab, et l'API. Voici les étapes nécessaires pour installer et faire fonctionner correctement l'ensemble :

Prérequis

Avant toute installation, il est nécessaire d'avoir les outils suivants installés sur la machine :

- PHP (version compatible avec Symfony)
- Symfony CLI
- Composer
- Node.js et npm

Étapes d'installation

Pour chaque projet Symfony (landing page, web et API), voici les étapes communes :

- Cloner le projet concerné
- Installer les dépendances PHP via Composer : composer install
- Installer les dépendances JavaScript : npm install
- Compiler les assets : npm run build
- Il est important de démarrer l'API en premier, car elle doit être accessible à l'adresse http://localhost:8000. Pour cela, il faut utiliser la commande suivante dans le répertoire de l'API : symfony server:start

Concernant l'application mobile, elle a été développée avec Flutter, afin de proposer une solution multiplateforme efficace et moderne.

Prérequis

Avant toute installation, il est nécessaire d'avoir les outils suivants installés sur la machine :

- Flutter SDK

- Emulateur android

Étapes d'installation

- Cloner le projet concerné
- Installer les dépendances avec la commande : flutter pub get
- Lancer l'application sur un simulateur ou un appareil : flutter run

4-API IGDB



4.1-Présentation de l'API

Pour alimenter Gamelab avec des données fiables et complètes sur les jeux vidéo, j'ai choisi d'utiliser l'API IGDB (Internet Game Database), une référence dans le domaine.

Qu'est-ce que l'API IGDB ?

L'API IGDB est une base de données en ligne fournie par Twitch, qui recense des informations détaillées sur des milliers de jeux vidéo : titres, plateformes, éditeurs, dates de sortie, genres, images, descriptions, etc. Elle est largement utilisée dans l'industrie pour sa richesse et sa fiabilité.

Pourquoi ce choix ?

- Base de données riche et à jour : IGDB propose des données très complètes, régulièrement mises à jour par la communauté et validées par Twitch.

- Large couverture : Elle couvre une grande variété de plateformes (PC, consoles, mobiles) et de types de jeux, ce qui répond parfaitement aux besoins de Gamelab.

- Accès structuré via GraphQL : L'API IGDB fonctionne avec GraphQL, ce qui permet de récupérer précisément les champs nécessaires et d'optimiser les requêtes réseau.

Intégration dans Gamelab

L'API IGDB est utilisée dans Gamelab pour :

- Rechercher des jeux à partir d'un mot-clé
- Afficher des fiches complètes (titre, visuel, description, plateformes, etc.)
- Proposer des suggestions et des filtres pertinents lors de l'ajout de jeux dans sa collection

4.2-Obtention des accès

Pour interagir avec l'API IGDB, une clé d'authentification fournie par Twitch est requise. Cette clé a été intégrée côté backend, via Symfony et API Platform, afin de centraliser les appels API, sécuriser les requêtes et filtrer les données avant leur transmission au front-end.

Ci-dessous, un extrait de mon fichier IGDBService.php, qui centralise l'ensemble des appels à l'API IGDB dans un service unique, garantissant une gestion plus propre, cohérente et maintenable des requêtes, dans ce fichier j'ai du importer mes accès afin de les utiliser pour faire les requêtes vers l'API IGDB :

```
class IgdbService

{

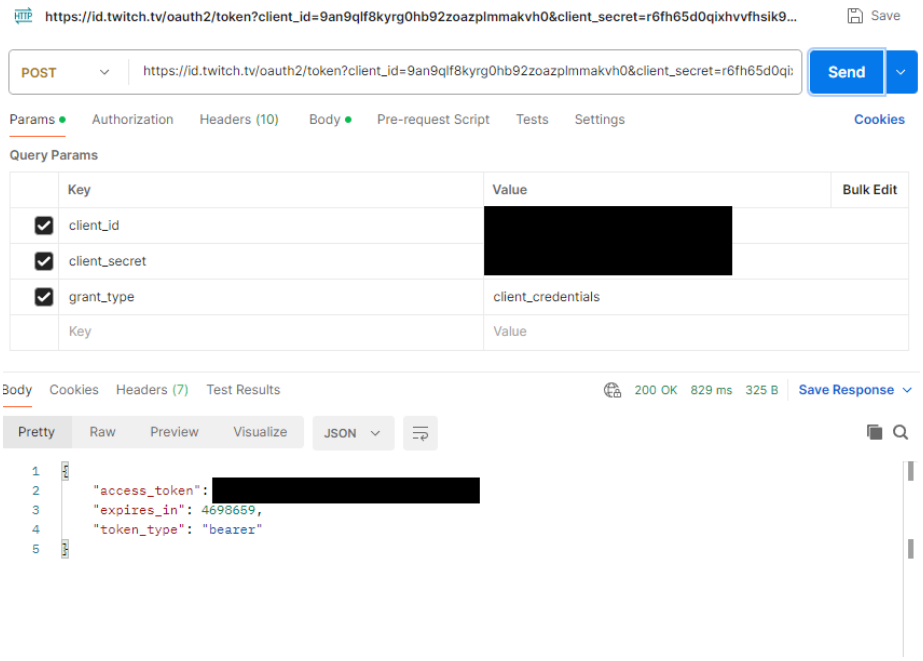
    private const BASE_URL = 'https://api.igdb.com/v4';

    private const CLIENT_ID = "MONIDCLIENT";

    private const ACCESS_TOKEN = "MACLEBEARER";
```

Pour obtenir cette clé, il a d'abord fallu créer un compte sur le Twitch Developer Portal et enregistrer mon application en précisant son nom et son objectif. Cette étape permet de générer un Client ID et un Client Secret nécessaires à l'authentification.

Cependant, ces identifiants seuls ne suffisent pas à interroger l'API IGDB. Il faut ensuite effectuer une requête d'authentification OAuth via Postman pour obtenir un token Bearer temporaire, indispensable pour accéder aux données de l'API.



Enfin je peux donc comme vu précédemment mettre mes identifiants dans mon service, mais attention, je dois les changer tous les 2 mois car l'api est sécurisé et la clé Bearer doit être renouvelée avec la même requête tous les 60 jours.

Extrait de la documentation de l'API de IGDB, section FAQ :

> **4. My AccessToken stopped working, why?**
>
> Your Access Token is only active for **60 days** and your application can only have **25 active** Access Tokens at one time, going over this limit starts to inactivate older tokens.

4.3-Test de l'API

Afin de valider le bon fonctionnement de l'API IGDB et de m'assurer de son intégration correcte dans le projet, j'ai réalisé un type de test pour savoir utiliser l'api de moi même :

- Test manuel via Postman : J'ai effectué des séries de tests pour trouver un jeu via son ID par exemple (Annexe 6)

Cette approche m'a permis de sécuriser l'intégration de l'API, tout en facilitant la détection d'erreurs potentielles à la fois pendant le développement et lors des mises à jour futures.

5-API Gamelab

5.1-Technologies utilisées

L'API a été développée à l'aide du framework Symfony, en s'appuyant sur la bibliothèque API Platform, qui simplifie considérablement la création d'API REST robuste, maintenable et bien structurée. Cette solution offre également la génération automatique d'une documentation complète, claire et interactive, ce qui accélère le développement tout en facilitant la compréhension du fonctionnement de l'API par d'autres développeurs.

La documentation générée repose sur Swagger UI, une interface front-end intégrée à API Platform, qui permet de consulter, tester et interagir facilement avec les différents endpoints de l'API.(Annexe 7)





5.2-Fonctionnalités principales de l'API

L'API de Gamelab constitue le cœur du traitement côté serveur. Elle prend en charge l'ensemble des fonctionnalités back-end nécessaires au bon fonctionnement de la plateforme. Parmi ses principales fonctionnalités, on retrouve :

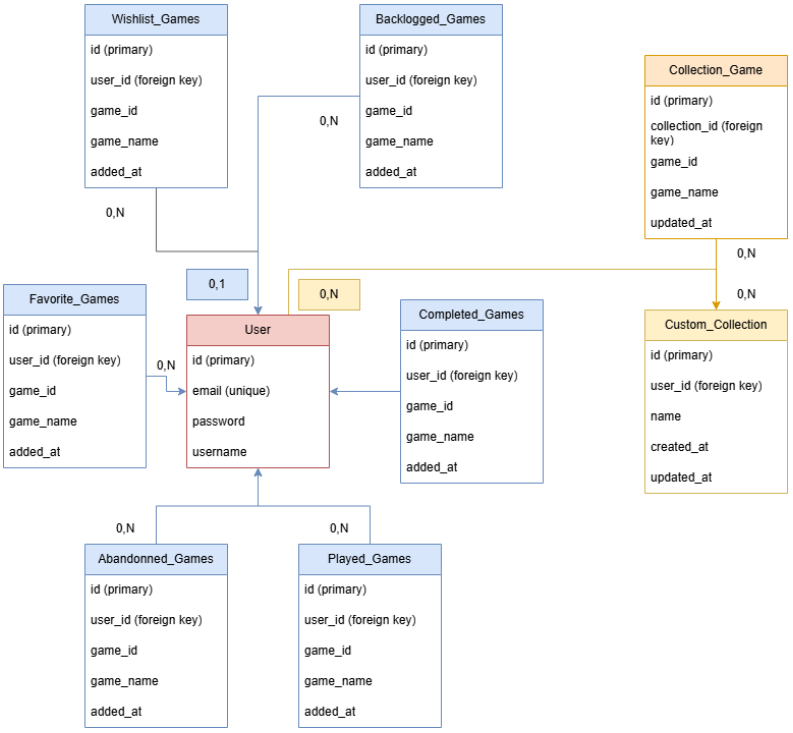
- Gestion des opérations CRUD (*Create, Read, Update, Delete*) sur les entités principales (collections, utilisateurs, etc.)
- Ajout de jeux dans les collections type favoris,joués, etc… pour permettre aux utilisateurs de constituer une liste rapide de leurs jeux préférés,joués etc…
- Création et gestion de collections personnalisées, permettant aux utilisateurs d'organiser leurs jeux selon leurs propres critères
- Gestion des comptes utilisateurs, avec les opérations de création, authentification et modification de profil
- Gestion du back office pour les administrateurs avec des endpoints personnalisées pour les administrateurs

L'API a été conçue de manière structurée et extensible afin de faciliter les évolutions futures du projet et l'ajout de nouvelles fonctionnalités.

5.3-Base données de l'application

La base de données de Gamelab repose sur MariaDB, un système de gestion de base de données relationnelle compatible avec SQL. Elle est utilisée pour stocker l'ensemble des données essentielles de l'application, telles que les utilisateurs, les jeux, les collections ou encore les favoris.

Pour la base de données j'ai créé un schéma MCD avant de faire toutes manipulations afin de pouvoir avoir une vue globale de l'application et ce que je pourrais sauvegarder en base de données sachant que je travaille avec une API Externe qui est IGDB.



Pour la gestion du schéma de base de données, j'ai utilisé Doctrine Migrations, un outil intégré à Symfony. Il permet de versionner les évolutions de la base sous forme de fichiers de migration (Annexe 8), et d'exécuter automatiquement les modifications (création ou mise à jour des tables, types de colonnes, relations, etc.) via des commandes Symfony.

Les migrations sont ensuite appliquées à la base MariaDB, assurant une synchronisation fiable entre le code et la structure de la base.

Enfin, pour visualiser et manipuler les données plus facilement pendant le développement, j'ai utilisé l'interface phpMyAdmin intégrée via mon XAMPP qui me sert à créer des serveurs locaux. (Annexe 9)

Logo Xampp

5.4-Gestion des utilisateurs

5.4.1-Inscription

Pour créer l'inscription, il était au préalable nécessaire de créer l'entité utilisateur, pour se faire j'ai effectué la commande : symfony console make:user, cette commande permet de créer l'entité utilisateur avec des paramètres pré-réglés comme par exemple le fait que l'email doit être unique. Cette commande est utilisable grâce au bundle (paquet) makerbundle, si il n'est pas initialisé pour quelconque raison il doit être installé via la commande : composer require --dev symfony/maker-bundle

```php
#[ORM\Id]
#[ORM\GeneratedValue]
#[ORM\Column]
#[Groups(['user:read', 'collection:read'])]
private ?int $id = null;

#[ORM\Column(length: 180, unique: true)]
#[Groups(['user:read', 'user:write'])]
#[Assert\NotBlank]
#[Assert\Email]
private ?string $email = null;


#[ORM\Column]
#[Groups(['user:read'])]
private array $roles = [];

#[ORM\Column]
private ?string $password = null;
```

Ensuite, j'ai développé un contrôleur personnalisé pour gérer les inscriptions via des requêtes JSON. Ce contrôleur vérifie les données reçues (email, mot de passe, pseudonyme), les valide, puis crée un nouvel utilisateur en base de données.

```php
#[Route('/api/register', name: 'app_register', methods: ['POST'])]
public function register(
    Request $request,
    EntityManagerInterface $entityManager,
    UserPasswordHasherInterface $passwordHasher,
    ValidatorInterface $validator
): JsonResponse {
    $data = json_decode($request->getContent(), true);

    $email = $data['email'] ?? null;
    $plainPassword = $data['plainPassword']   ?? $data['password']   ?? null;
    $username      = $data['name']            ?? $data['username']   ?? null;

    if (!$email || !$plainPassword || !$username) {
        return new JsonResponse(['error' => 'Données manquantes'], JsonResponse::HTTP_BAD_REQUEST);
    }
```

J'y ai également intégré un système de hachage des mots de passe à l'aide du service UserPasswordHasherInterface de Symfony. Cela permet de sécuriser les mots de passe en les enregistrant sous forme chiffrée, et non en claire, ce qui est essentiel pour protéger les données sensibles des utilisateurs.

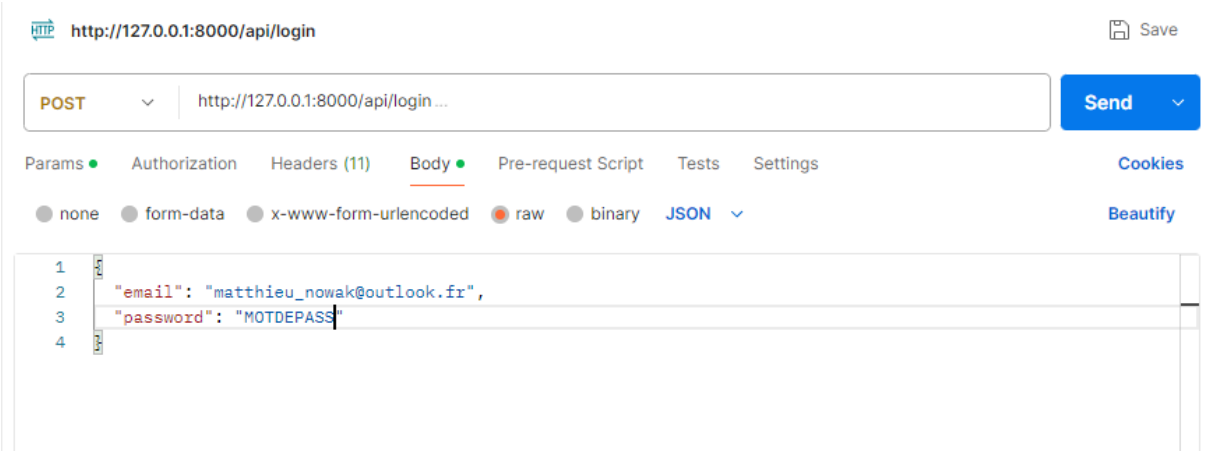
```php
// Hachage à partir de la clé $plainPassword
$hashed = $passwordHasher->hashPassword($user, $plainPassword);
$user->setPassword($hashed);
```

```php
// Le mot de passe en clair , non persisté en base de données.
#[Groups(['user:write'])]
#[Assert\NotBlank(groups: ['create'])]
#[Assert\Length(
    min: 12,
    minMessage: 'Le mot de passe doit contenir au moins 12 caractères'
)]
#[Assert\Regex(
    pattern: '/[A-Z]/',
    message: 'Le mot de passe doit contenir au moins une lettre majuscule'
)]
#[Assert\Regex(
    pattern: '/\d/',
    message: 'Le mot de passe doit contenir au moins un chiffre'
)]
#[Assert\Regex(
    pattern: '/[\W_]/',
    message: 'Le mot de passe doit contenir au moins un caractère spécial'
)]
private ?string $plainPassword = null;
```

5.4.2-Connexion

Le système de connexion a été mis en place via un contrôleur personnalisé appelé SecurityController (Annexe 10), accessible à l'URL /api/login. Il permet d'authentifier un utilisateur à partir de ses identifiants (email et mot de passe) et d'établir une session sécurisée.

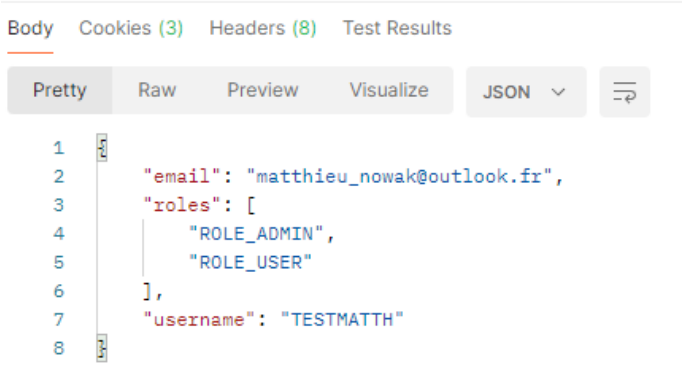
Lorsqu'un utilisateur envoie une requête POST à /api/login, Symfony vérifie automatiquement ses identifiants grâce à l'annotation #[CurrentUser]. Si les identifiants sont valides, l'utilisateur est authentifié.



Ensuite, le contrôleur :

- Créer un token d'authentification avec UsernamePasswordToken.
- Stocke ce token dans le TokenStorageInterface pour maintenir la session active.
- Initialise la session Symfony, ce qui permet de conserver l'utilisateur connecté lors des futures requêtes.

En réponse, l'API renvoie les informations de l'utilisateur (email, rôle, pseudo) au format JSON.



L'accès aux données utilisateurs est protégé par des règles de sécurité définies directement dans l'entité User grâce aux annotations d'API Platform. Ces règles empêchent par exemple un utilisateur de voir, modifier ou supprimer le compte d'un autre.

```php
#[ORM\Entity(repositoryClass: UserRepository::class)]
#[ORM\UniqueConstraint(name: 'UNIQ_IDENTIFIER_EMAIL', fields: ['email'])]
#[ApiResource(
    normalizationContext: ['groups' => ['user:read']],
    denormalizationContext: ['groups' => ['user:write']],
    operations: [
        new GetCollection(security: "is_granted('ROLE_ADMIN')"), // liste des utilisateurs (admin seulement)
        new Post(),          // création d'un utilisateur
        new Get( // détails d'un utilisateur
            security: "object == user or is_granted('ROLE_ADMIN')",
            securityMessage: "Vous ne pouvez consulter que vos propres informations ou être administrateur."
        ),
        new Put( // modification d'un utilisateur
            security: "object == user or is_granted('ROLE_ADMIN')",
            securityMessage: "Vous ne pouvez modifier que vos propres informations ou être administrateur."
        ),
        new Delete( // suppression d'un utilisateur
            security: "object == user or is_granted('ROLE_ADMIN')",
            securityMessage: "Vous ne pouvez supprimer que votre propre compte ou être administrateur."
        )
    ]
)]
```

```php
#[Groups(['user:read', 'collection:read'])]
private ?int $id = null;
```

Une fois connecté, l'utilisateur reçoit un cookie de session, ce qui lui permet de rester authentifié sans avoir à renvoyer ses identifiants à chaque requête.

```php
    public function authenticate(Request $request): Passport
    {
        // Extraire le token JWT de la requête
        $token = $this->tokenExtractor->extract($request);

        if (null === $token) {
            // Si aucun token n'est trouvé, lancer une exception
```

```
            throw new CustomUserMessageAuthenticationException('No JWT
token found');
        }

        try {
            // Valider le token et extraire les données utilisateur
            $userData = $this->jwtManager->parse($token);

            // Créer un UserBadge avec les données utilisateur
            return new SelfValidatingPassport(
                                new  UserBadge($userData['username']  ??
$userData['email'], function ($userIdentifier) {
                                                                    return
$this->userProvider->loadUserByIdentifier($userIdentifier);
                })
            );
        } catch (\Exception $exception) {
             throw new CustomUserMessageAuthenticationException('Invalid
JWT Token');
        }
    }
```

Enfin, Une route POST /api/logout permet à l'utilisateur de se déconnecter. Elle invalide la session en cours et supprime le token d'authentification, assurant ainsi une fin de session propre et sécurisée.

```
        #[Route  (path:  '/api/logout',  name:  'api_logout',  methods:
['POST'])]
    public function logout(TokenStorageInterface $tokenStorage, Request
$request){
        // Session invalide
        $session = $request->getSession();
        $session->invalidate();

        // Supprimer le token de sécurité
        $tokenStorage->setToken(null);

            return  new  JsonResponse(['message'  =>  'Vous  avez  été
déconnecté'], JsonResponse::HTTP_OK);
    }
```

5.5-Fonctionnalités Principales

5.5.1-Ajout dans une collection (Favoris)

L'ajout d'un jeu dans la collection des favoris d'un utilisateur est géré via un contrôleur dédié, FavoriteGameController. Ce contrôleur permet d'ajouter, lister, vérifier ou supprimer un jeu favori via des endpoints sécurisés accessibles uniquement aux utilisateurs connectés.(Annexe 11)

L'ajout se fait par une requête POST envoyée à l'URL /api/favorites, contenant l'identifiant du jeu (gameId). Voici les étapes principales du traitement :

- Récupération de l'utilisateur courant (via getUser()).
- Validation de l'ID du jeu envoyé dans le corps de la requête JSON.
- Vérification des doublons : si le jeu est déjà dans les favoris de l'utilisateur, une erreur est retournée.
- Récupération du nom du jeu via un appel à l'API IGDB à l'aide d'un service dédié (IgdbService).
- Création d'un nouvel objet FavoriteGame avec l'utilisateur, l'ID et le nom du jeu.
- Enregistrement dans la base de données avec Doctrine.

Le contrôleur est protégé par l'annotation #[IsGranted('ROLE_USER')], ce qui signifie que seuls les utilisateurs authentifiés peuvent accéder à ces routes.

5.5.2-Création d'une collection personnalisée

Dans le cadre du projet, une fonctionnalité a été mise en place pour permettre aux utilisateurs authentifiés de créer leurs propres collections personnalisées de jeux. Cette fonctionnalité offre à chaque utilisateur la possibilité de regrouper les jeux de son choix selon des critères personnels, comme ses genres préférés, ses envies du moment ou encore ses titres à terminer.

La création d'une collection passe par un appel à l'API via la méthode POST à l'URL /api/collections. Le corps de la requête doit contenir le nom de la collection souhaitée. Cette dernière est ensuite enregistrée en base de données avec les informations de l'utilisateur qui l'a créée, ainsi que la date de création.

L'entité CustomCollection a été développée pour représenter cette collection dans le système. Elle contient notamment les propriétés suivantes : un identifiant unique, un nom, une relation avec l'utilisateur, une date de création et une liste de jeux associés.

Le contrôleur CustomCollectionController gère la logique métier liée à cette fonctionnalité. Il vérifie que l'utilisateur est bien connecté (grâce au rôle ROLE_USER) et que les données transmises sont valides. Si tout est conforme, une nouvelle collection est persistée en base de données via l'EntityManager de Symfony.

5.5.3-Notation d'un jeu

Le système de notation permet aux utilisateurs de laisser une note personnelle (de 0.5 à 5, par paliers de 0.5) sur les jeux de leur choix, accompagnée d'un commentaire facultatif. Il est entièrement géré par le contrôleur GameRatingController, et accessible uniquement aux utilisateurs connectés.

L'utilisateur envoie une requête POST à /api/ratings contenant :

- gameId : l'identifiant du jeu
- rating : la note attribuée (entre 0.5 et 5, par pas de 0.5)
- comment (optionnel) : un commentaire lié à la note

Le contrôleur va quant à lui :

- Valider les données (ID du jeu et valeur de la note)
- Vérifier si l'utilisateur a déjà noté ce jeu
- Récupérer le nom du jeu via l'API IGDB
- Enregistrer la note dans la base de données via Doctrine

Les routes liées à l'ajout, la suppression et la consultation privée de notes sont protégées par l'annotation #[IsGranted('ROLE_USER')], garantissant que seuls les utilisateurs authentifiés peuvent interagir avec le système de notation.

5.5.4-Système d'aministration (Back-office)

Pour le système de back-office en API c'est assez simple, j'ai tout d'abord permis aux utilisateurs qui ont le rôle 'ROLE_ADMIN' de pouvoir manipuler les entités, grâce à "is_granted" en CRUD, par exemple avec l'entité User.php :

```php
#[ApiResource(
    normalizationContext: ['groups' => ['user:read']],
    denormalizationContext: ['groups' => ['user:write']],
    operations: [
        new GetCollection(security: "is_granted('ROLE_ADMIN')"), // liste des utilisateurs (admin seulement)
        new Post(),          // création d'un utilisateur
        new Get( // détails d'un utilisateur
            security: "object == user or is_granted('ROLE_ADMIN')",
            securityMessage: "Vous ne pouvez consulter que vos propres informations ou être administrateur."
        ),
        new Put( // modification d'un utilisateur
            security: "object == user or is_granted('ROLE_ADMIN')",
            securityMessage: "Vous ne pouvez modifier que vos propres informations ou être administrateur."
        ),
        new Delete( // suppression d'un utilisateur
            security: "object == user or is_granted('ROLE_ADMIN')",
            securityMessage: "Vous ne pouvez supprimer que votre propre compte ou être administrateur."
        )
    ]
)]
```

Et dans le UserController afin de pouvoir manipuler les CRUD j'ai créé des routes personnalisées, par exemple pour voir les utilisateurs :

```php
#[Route('/users', name: 'api_users_list', methods: ['GET'])]
#[IsGranted('ROLE_ADMIN')]
public function list(): JsonResponse
{
    $users = $this->userRepository->findAll();

    return $this->json([
        'users' => array_map(function($user) {
            return [
                'id' => $user->getId(),
                'email' => $user->getEmail(),
                'username' => $user->getUsername(),
                'roles' => $user->getRoles()
            ];
        }, $users)
    ]);
}
```

La fonction getUserById permet de récupérer les informations d'un utilisateur en fonction de son identifiant (id). Elle utilise le repository pour chercher l'utilisateur dans la base de données. Si aucun utilisateur n'est trouvé avec cet identifiant, elle retourne une réponse JSON avec un message d'erreur et le code HTTP 404. Sinon, elle retourne un tableau contenant les informations principales de l'utilisateur (id, email, username, roles) au format JSON.

Et pour les supprimer :

```php
#[Route('/users/{id}', name: 'api_users_delete', methods: ['DELETE'])]
#[IsGranted('ROLE_ADMIN')]
public function deleteUser(int $id): JsonResponse
{
    $user = $this->userRepository->find($id);

    if (!$user) {
        return $this->json(['error' => 'Utilisateur non trouvé'], Response::HTTP_NOT_FOUND);
    }

    // Vérifier que l'admin ne se supprime pas lui-même
    if ($user === $this->getUser()) {
        return $this->json(['error' => 'Vous ne pouvez pas supprimer votre propre compte via cette API'], Response::HTTP_FORBIDDEN);
    }

    $this->entityManager->remove($user);
    $this->entityManager->flush();

    return $this->json(['message' => 'Utilisateur supprimé avec succès']);
}
```

La fonction deleteUser permet de supprimer un utilisateur à partir de son identifiant (id). Elle commence par rechercher l'utilisateur dans la base de données. Si l'utilisateur n'existe pas, elle renvoie une erreur 404. Ensuite, elle vérifie que l'administrateur connecté ne tente pas de supprimer son propre compte, ce qui est interdit pour éviter des erreurs critiques — si c'est le cas, une erreur 403 est retournée. Si toutes les conditions sont remplies, l'utilisateur est supprimé, et une réponse JSON confirme la suppression avec un message de succès.

Et ainsi de suite pour les autres opérations modifier et créer, cela me permet maintenant d'avoir un back office fonctionnel et sécurisé grâce à IsGranted qui n'est que accessible pour les utilisateurs ayant le rôle admin.

5.6-Test Unitaire

5.6.1-Création d'un utilisateur et connexion

Les fonctionnalités telles que l'inscription et la connexion sont essentielles pour permettre à un nouvel utilisateur d'accéder à toutes les fonctionnalités de Gamelab. Un test fonctionnel a été mis en place pour valider ces étapes dans leur intégralité :

```php
public function testRegistrationAndLogin(): void
{
    try {
        // Étape 1: Inscription
        $this->client->request('POST', '/api/register', [], [], [
            'CONTENT_TYPE' => 'application/json'
        ], json_encode([
            'email' => $this->testEmail,
            'plainPassword' => 'StrongPassword123!@#',
            'name' => 'User Test'
        ]));

        $registerResponse = $this->client->getResponse();
        $registerStatusCode = $registerResponse->getStatusCode();
        $registerData = json_decode($registerResponse->getContent(), true);
        $userAlreadyExists = $registerStatusCode === Response::HTTP_BAD_REQUEST &&
                             isset($registerData['errors']) &&
                             is_array($registerData['errors']) &&
                             count(array_filter($registerData['errors'], function($error) {
                                 return strpos($error, 'déjà utilisé') !== false ||
                                        strpos($error, 'already used') !== false;
                             })) > 0;

        if ($registerStatusCode !== Response::HTTP_CREATED && !$userAlreadyExists) {
            $this->assertEquals(Response::HTTP_CREATED, $registerStatusCode,
                "L'inscription a échoué (code attendu: 201, reçu: $registerStatusCode)");
        }

        // Étape 2: Connexion avec les identifiants créés
        $this->client->request('POST', '/api/login', [], [], [
            'CONTENT_TYPE' => 'application/json'
        ], json_encode([
            'email' => $this->testEmail,
            'password' => 'StrongPassword123!@#'
        ]));

        $response = $this->client->getResponse();
        $statusCode = $response->getStatusCode();
        $responseData = json_decode($response->getContent(), true);

        $this->assertEquals(Response::HTTP_OK, $statusCode,
            "La connexion a échoué après inscription (code attendu: 200, reçu: $statusCode). " .
            "Réponse: " . $response->getContent());
        $this->assertArrayHasKey('email', $responseData, "La réponse ne contient pas le champ 'email'");
        $this->assertEquals($this->testEmail, $responseData['email']);
    } catch (\Exception $e) {
        $this->markTestIncomplete('Exception lors du test d\'inscription et connexion: ' . $e->getMessage());
    }
}
```

Le test utilise le client HTTP de Symfony pour simuler deux requêtes consécutives :

- Une requête POST /api/register pour créer un compte utilisateur.
- Une requête POST /api/login pour se connecter avec les identifiants créés.

Le scénario combine deux actions :

- L'inscription d'un utilisateur avec un email de test différent
- La connexion avec ces mêmes identifiants via l'endpoint /api/login.

Le test prend en compte le cas où l'utilisateur existe déjà. Dans ce cas, il ignore l'erreur d'inscription et procède directement à la tentative de connexion.

Le test vérifie alors que :

- La connexion retourne un code 200 (OK) ;
- Le champ email est bien présent dans la réponse JSON et correspond à l'adresse utilisée.

Ce scénario confirme que l'utilisateur peut se connecter immédiatement après avoir été inscrit, ce qui est une exigence fonctionnelle critique.

On peut en conclure, que ce test permet de garantir la robustesse du mécanisme d'authentification de l'API en validant la création et l'utilisation d'un compte utilisateur dans un scénario complet.

5.6.3-Ajout d'un jeu en favoris

L'ajout d'un jeu en favoris est une fonctionnalité phare de Gamelab, car elle sert de base pour d'autres systèmes similaires comme les jeux souhaités, ou déjà joués. Le fonctionnement est quasiment identique pour toutes ces collections : seul le type de liste ciblée change. Cela permet de centraliser la logique côté backend et de la dupliquer facilement selon les besoins.

Le test mis en place simule un parcours utilisateur classique :

- Inscription d'un utilisateur test
- Connexion de cet utilisateur
- Ajout d'un jeu dans ses favoris via une requête POST sur l'adresse /api/favorites

Cette suite d'actions permet de vérifier que le système fonctionne de bout en bout, avec une session bien maintenue et une interaction fluide avec l'API.

Le test utilise Symfony pour envoyer des requêtes API simulées. Un faux jeu (ID 12345, nom "Super Test Game") est injecté dans le système en remplaçant temporairement le service IGDB. Ce "mock" évite de faire un vrai appel à l'API externe tout en testant le comportement attendu.

Ensuite, l'utilisateur est enregistré et connecté. Une fois connecté, il envoie une requête pour ajouter le jeu dans ses favoris. Le test vérifie que tout se passe correctement à chaque étape.

Voici la requête d'ajout au favoris et les vérifications attendues dans la réponse. Il peut être réutilisé presque à l'identique pour d'autres listes comme "souhaités" ou "joués".

```
            $client->request(
                'POST',
                '/api/favorites',
                [],
                [],
                ['CONTENT_TYPE' => 'application/json'],
                json_encode(['gameId' => 12345])
            );
```

```
            $favoriteResponse = $client->getResponse();
            $favoriteStatusCode = $favoriteResponse->getStatusCode();

                                $this->assertEquals(Response::HTTP_CREATED,
$favoriteStatusCode,
                "L'ajout aux favoris a échoué (code attendu: 201, reçu:
$favoriteStatusCode). " .
                "Réponse: " . $favoriteResponse->getContent());

              $data = json_decode($client->getResponse()->getContent(),
true);
            $this->assertArrayHasKey('id', $data);
            $this->assertSame(12345, $data['gameId']);
            $this->assertSame('Super Test Game', $data['gameName']);
```

5.6.3-Test non fonctionnel

En plus des tests classiques qui permettent une validation des résultats, il est impératif de tester des erreurs intentionnelles pour vérifier si l'api renvoie bien une erreur quand elle doit en envoyer une.

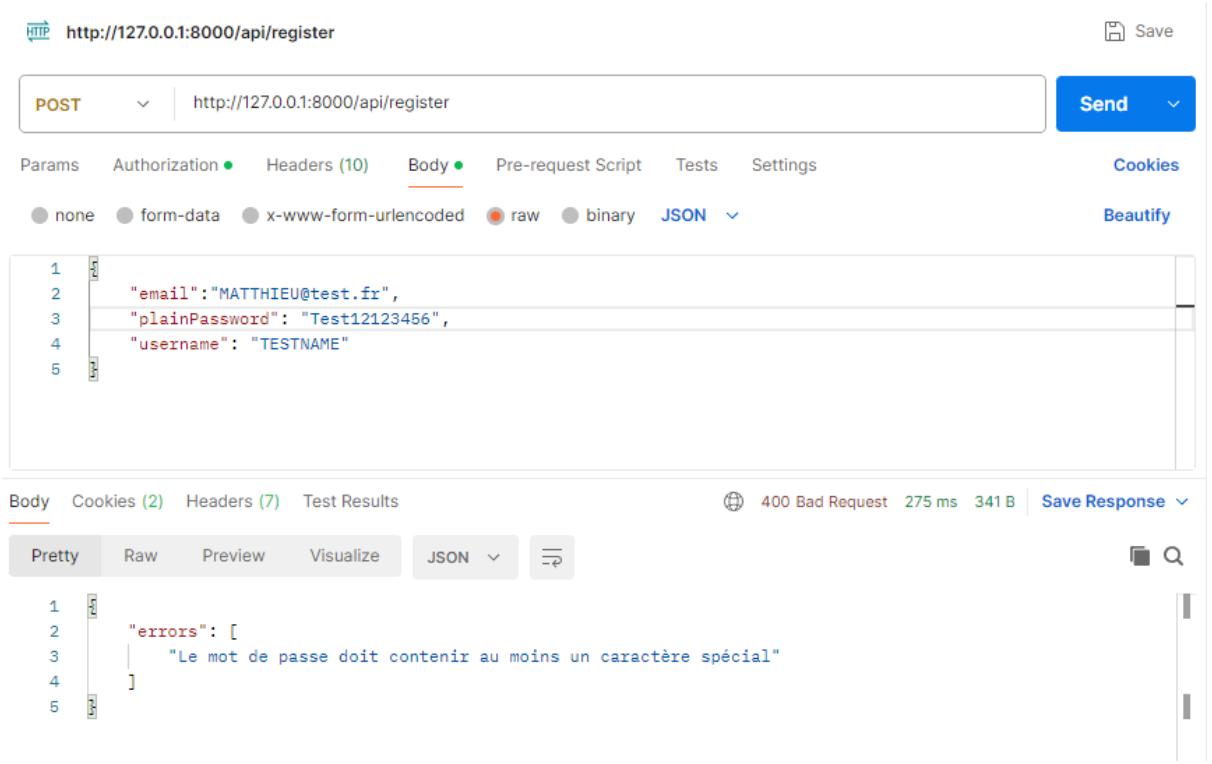
Un des tests non fonctionnel est d'empêcher un utilisateur de s'enregistrer s'il ne respecte pas les contraintes de mot de passe imposé (REGEX) qui sont les suivantes :

```
    #[Groups(['user:write'])]
    #[Assert\NotBlank(groups: ['create'])]
    #[Assert\Regex(
        pattern: '/^(?=.*[A-Z])(?=.*\d)(?=.*[\W_]).{12,}$/',
         message: 'Le mot de passe doit contenir au moins 12 caractères,
une majuscule, un chiffre et un caractère spécial.'
    )]
    private ?string $plainPassword = null;
```

J'ai donc tester le même test que le 5.6.1 à la seule différence que j'ai mis un mot de passe ne respectant pas la longueur imposé par le REGEX voici le résultat obtenu :

```
1) App\Tests\Functional\RegistrationFunctionalFailedTest::testRegistrationWithInvalidPassword
L'inscription avec un mot de passe trop court devrait échouer avec un code 400
Failed asserting that 201 matches expected 400.
```

Et quand j'effectue un test manuel via Postman, j'obtiens :



Le test effectué est donc bel et bien concluant et me met une erreur si je ne respecte pas les conditions imposées dans le mot de passe.

5.7 – Dockerisation de l'API

Dans un souci de professionnalisation du développement, de portabilité et de standardisation de l'environnement d'exécution, la dockerisation de l'API Gamelab a été mise en place. Cette démarche vise à garantir un fonctionnement homogène de l'application, quel que soit le poste de travail ou l'environnement serveur utilisé.

L'utilisation de Docker répond à plusieurs besoins :

- Isolation de l'environnement : chaque conteneur embarque ses propres dépendances, évitant ainsi les conflits entre bibliothèques ou versions.

- Facilité de mise en place : un nouvel utilisateur peut lancer le projet sans configuration complexe.

- Cohérence entre les environnements de développement, de test et de production.

- Préparation à un futur déploiement sur une infrastructure cloud ou un hébergement dédié.

Le fichier Dockerfile situé à la racine du projet permet de construire une image légère et optimisée de l'API :

```yaml
services:
  php:
    image: php:8.2-cli
    container_name: gamelab_php
    volumes:
      - .:/var/www/symfony
    depends_on:
      - database
    working_dir: /var/www/symfony
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: "mysql://root:password@database:3306/gamelab?serverVersion=mariadb-10.4.28"
    command: |
      bash -c "apt-get update &&
      apt-get install -y git unzip libzip-dev &&
      docker-php-ext-install pdo pdo_mysql zip &&
      curl -sS https://getcomposer.org/installer | php -- --install-dir=/usr/local/bin --filename=composer &&
      composer install &&
      echo 'Lancement du serveur PHP...' &&
      php -S 0.0.0.0:8000 -t public"

  database:
    image: mariadb:10.4.28
    container_name: gamelab_db
    environment:
      MYSQL_ROOT_PASSWORD: password
      MYSQL_DATABASE: gamelab
      MYSQL_USER: app
      MYSQL_PASSWORD: password
    ports:
      - "3306:3306"
    volumes:
      - database_data:/var/lib/mysql:rw

  phpmyadmin:
    image: phpmyadmin/phpmyadmin
    container_name: gamelab_phpmyadmin
    depends_on:
      - database
    ports:
      - "8080:80"
    environment:
      PMA_HOST: database
      MYSQL_ROOT_PASSWORD: password
      PMA_USER: root
      PMA_PASSWORD: password
```

Ce fichier docker-compose.yaml permet de lancer une stack complète pour un projet PHP avec Symfony.

Il crée trois services :

- un conteneur PHP (gamelab_php) qui exécute l'application Symfony
- un conteneur MariaDB (gamelab_db) qui héberge la base de données
- et un conteneur PhpMyAdmin (gamelab_phpmyadmin) pour visualiser et gérer les données via une interface web.

Le conteneur PHP installe automatiquement les dépendances nécessaires (extensions PHP, Composer, etc.) et démarre le serveur web. Chaque service est configuré avec les variables d'environnement nécessaires à son bon fonctionnement, et les ports sont exposés pour un accès local (API sur le port 8000, PhpMyAdmin sur le 8080). Ce setup permet d'avoir un

environnement de développement complet, prêt à l'emploi, avec une base de données connectée et une interface d'administration.

6-Landing Page

6.1-Maquette

Pour concevoir la landing page de Gamelab, qui a pour objectif d'expliquer l'application ainsi que les différentes façons d'y accéder (via le web ou en téléchargeant l'APK), j'ai d'abord réalisé une maquette de celle-ci. J'ai utilisé Figma pour le prototypage, en m'inspirant de la page d'accueil du site Backloggd, notamment pour la disposition en arrière-plan des jaquettes de jeux (Annexe 12). J'ai toutefois intégré un système d'affichage aléatoire des jaquettes, que je détaillerai dans le chapitre 6.2.

Concernant le personnage au premier plan, je me suis appuyé sur une maquette communautaire Figma du jeu Spider-Man: Miles Morales, dont j'ai adapté le style à l'univers de Gamelab. (Annexe 13)

J'ai donc combiné le personnage en premier plan et les jaquettes de jeux avec un système aléatoire pour avoir un beau design interactif (Annexe 14)

6.2-Design basé sur l'aléatoire

Dans la Landing page de Gamelab, j'ai intégré un design basé sur l'aléatoire dans la section d'en-tête appelée hero header. Ce concept a pour but de rendre l'interface plus dynamique et engageante, en proposant à chaque chargement de la page un contenu visuel différent.

Le design aléatoire repose ici sur l'affichage de trois lignes d'images de jeux différentes, définies par les variables gamesLine1, gamesLine2 et gamesLine3. Ces lignes sont générées aléatoirement dans le contrôleur au moment du rendu de la page d'accueil.

Dans le fichier hero_header.html.twig, chaque ligne de jeux est affichée via une boucle for :

```
{# Section 1 #}
    <section class="games-section">
      <div class="games-container">
        {% for game in gamesLine1 %}
          <div class="game-item">
```

```
            {% if game.cover is defined and game.cover.url is defined
%}
              <img
                src="{{
game.cover.url|replace({'t_thumb':'t_cover_big'}) }}"
                alt="{{ game.name }}"
                class="game-cover"
                height="100px"
                width="100px"
              >
            {% endif %}
          </div>
        {% endfor %}
      </div>
    </section>
```

En plus des jeux, j'ai également intégré l'affichage aléatoire d'un personnage dans la section de droite du hero_header. Ce personnage est sélectionné de manière aléatoire à chaque chargement de la page, ce qui contribue à varier l'aspect visuel et à renforcer l'identité de la plateforme :

```
      <div class="flex justify-end">
        <img
          src="{{ character }}"
          alt="Featured Character"
          class="object-contain max-h-[600px]"
        >
      </div>
```

La partie aléatoire est directement gérée dans mon HomeController qui fait des appels de l'API Igdb pour les jaquettes des jeux vidéos et qui vient chercher dans mon dossier personnage un personnage aélatoire (Annexe 15)

7-Application Web

7.1-Récupération des différentes API et utilisation

Pour intégrer les données issues des API Gamelab et IGDB dans mon application web, j'ai développé deux services distincts :

- "GamelabApiService.php" pour l'API interne de Gamelab
- "IgdbService.php" pour l'API externe d'IGDB, accessible via Twitch

7.1.1 Service de l'api Gamelab

Ce service est dédié à la communication avec l'API Gamelab, généralement utilisée pour récupérer les données propres à l'environnement de travail, comme les utilisateurs ou encore récupérer les jeux de différentes collections. Voici un extrait du constructeur :

```php
public function __construct(RequestStack $requestStack, IgdbService $igdbService)
{
    $this->requestStack = $requestStack;
    $this->igdbService  = $igdbService;
    $this->httpClient   = HttpClient::create([
        'headers' => [
            'X-API-KEY'    => self::API_KEY,
            'Content-Type' => 'application/json',
            'Accept'       => 'application/json',
        ],
    ]);
}
```

Ce constructeur initialise trois éléments essentiels au fonctionnement du service : RequestStack permet d'accéder à la requête HTTP courante, ce qui est utile notamment pour la gestion des sessions.

IgdbService est injecté afin de pouvoir combiner les données issues des API IGDB et Gamelab si nécessaire.

Enfin, un client HTTP personnalisé est créé avec des en-têtes par défaut tels que X-API-KEY pour l'authentification, ainsi que Content-Type et Accept pour indiquer que les échanges se font en JSON. Cela permet de centraliser la configuration des requêtes API et d'éviter toute redondance.

Après le constructeur, les différentes requêtes vers l'endpoint sont mis dans ce service, un exemple d'utilisation concrète de ce service est la méthode toggleFavorite :

```php
public function toggleFavorite(int $gameId, bool $add = true): array
{
    $this->refreshTokenFromSession();

    if ($add) {
        $url = self::API_BASE_URL . '/api/favorites';
        $method = 'POST';
        $game = $this->igdbService->fetchGameDetails($gameId);
        $options = [
            'json' => ['gameId' => $gameId]
        ];
    } else {
        $url = self::API_BASE_URL . "/api/favorites/{$gameId}";
        $method = 'DELETE';
        $options = [];
    }

    try {
        $response = $this->httpClient->request($method, $url, $options);
        $status = $response->getStatusCode();

        if (in_array($status, [200, 201], true)) {
            $data = $response->toArray(false);
            return ['success' => true, 'data' => $data];
        }

        return ['success' => false, 'error' => 'Statut de réponse inattendu: ' . $status];
    } catch (\Throwable $e) {
        return ['success' => false, 'error' => $e->getMessage()];
    }
}
```

La fonction permet à un utilisateur connecté d'ajouter ou de retirer un jeu de ses favoris. Lorsqu'elle est appelée, cette méthode commence par actualiser le client HTTP avec les bons cookies de session grâce à la méthode "refreshTokenFromSession" afin de garantir une authentification valide.

Ensuite, si l'on souhaite ajouter un jeu, une requête POST est envoyée vers l'endpoint /api/favorites avec l'identifiant du jeu dans le corps de la requête. Avant cela, les détails du jeu sont récupérés via le service IgdbService, ce qui permet de croiser les informations issues d'IGDB avec celles de l'API Gamelab.

En revanche, si l'utilisateur souhaite retirer un jeu de ses favoris, une requête DELETE est effectuée vers /api/favorites/{gameId}. Dans les deux cas, la réponse de l'API est analysée pour en vérifier le statut HTTP : si le code indique un succès (200 ou 201), les données sont renvoyées à l'interface ; sinon, un message d'erreur est retourné.

7.1.1 Service de l'api IGDB

Le service IgdbService permet de communiquer avec l'API IGDB (Internet Game Database), une base de données très complète dédiée aux jeux vidéo, aux plateformes, aux studios ou encore aux genres. Contrairement à l'API Gamelab, IGDB est une API tierce, appartenant à Twitch, qui nécessite une authentification via un identifiant client (Client-ID) et un jeton d'accès (Bearer Token), transmis via les en-têtes HTTP.

Voici le constructeur du service :

```php
public function __construct()
{
    $this->httpClient = HttpClient::create([
        'headers' => [
            'Client-ID'       => self::CLIENT_ID,
            'Authorization'   => 'Bearer ' . self::ACCESS_TOKEN,
            'Content-Type'    => 'text/plain',
            'Accept'          => 'application/json',
        ],
    ]);
}
```

Ce constructeur initialise un client HTTP en précisant les en-têtes attendus par l'API IGDB. À noter que le Content-Type est défini à text/plain, car IGDB attend les requêtes dans un format spécifique, qui ressemble à une syntaxe de type SQL ou GraphQL, et non pas du JSON standard. Cela permet d'exécuter des requêtes très ciblées et optimisées.

Les appels à IGDB se font ensuite via une méthode interne makeRequest, qui gère les requêtes POST vers un endpoint donné. Voici un exemple de son utilisation à travers la méthode fetchComingSoonGames :

```php
/**
 * Récupère les jeux à venir.
 *
 * @param int $offset
 * @return Game[]
 * @throws \Exception
 */
public function fetchComingSoonGames(int $offset = 0): array
{
    $currentTimestamp = time();
    $query = <<<EOT
        fields id,name,cover.url;
        where first_release_date > {$currentTimestamp};
        limit 50;
        offset {$offset};
        sort first_release_date asc;
        EOT;
    $data = $this->makeRequest('/games', $query);
    $games = [];
    foreach ($data as $gameData) {
        $games[] = Game::fromArray($gameData);
    }
    return $games;
}
```

Cette méthode permet de récupérer une liste de jeux dont la date de sortie est supérieure à la date actuelle, donc des jeux à venir. Elle commence par générer un timestamp UNIX pour représenter le moment présent, puis construit une requête en texte brut qui sera envoyée à

l'endpoint /games de l'API IGDB. Les champs demandés ici sont id, name et cover.url pour afficher les éléments essentiels du jeu dans l'interface. Le tri est fait par date de sortie croissante, et la pagination est contrôlée via un offset.

Ce type de requête met en lumière la puissance de l'API IGDB, qui permet de cibler avec précision les données nécessaires sans surcharge inutile. Une fois les données reçues, elles sont transformées en objets Game grâce à la méthode Game::fromArray, ce qui permet une manipulation plus propre dans le reste de l'application Symfony.

7.2-Back-office

Le back-office de l'application Gamelab est uniquement disponible sur la version web. J'ai fait ce choix car, selon moi, modérer du contenu ou gérer des utilisateurs n'est pas pratique sur un téléphone. Ce genre de tâches demande plus de visibilité, de confort et de précision, ce qui est plus adapté à un écran d'ordinateur. Du coup, toute la partie administration est prévue pour être utilisée sur un navigateur, avec une interface claire et plus facile à prendre en main.

Comme vu précédemment (page #) le back office est uniquement accessible via le rôle administrateur. De ce fait j'ai fais une condition dans le header de mon application web, si on est administrateur on peut y accéder depuis le dropdown de l'utilisateur là où se trouve le bouton de déconnexion.

```
{% if app.session.get('user') is not null and app.session.get('user')['roles'] is defined and 'ROLE_ADMIN' in app.session.get('user')['roles'] %}
    <a href="{{ path('app_admin_dashboard') }}" class="block px-4 py-2 text-sm text-[#272727] hover:bg-gray-100">
        <div class="flex items-center">
            <svg xmlns="http://www.w3.org/2000/svg" class="h-5 w-5 mr-2" fill="none" viewBox="0 0 24 24" stroke="currentColor">
                <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M10.325 4.317c.426-1.756 2.924-1.756 3.35 0a1.724 1.724 0 002.573 1.0
                <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M15 12a3 3 0 11-6 0 3 3 0 016 0z" />
            </svg>
            Administration
        </div>
    </a>
{% endif %}
```

Une fois cliqué sur le bouton, l'utilisateur arrive sur le back-office, directement sur la page dashboard. (Annexe 16)

Si par exemple il clique sur le bouton vers l'administration des utilisateurs il tombera sur la gestion des utilisateurs avec les différents boutons CRUD des utilisateurs (Annexe 17)

Sur cette page, l'administrateur peut visualiser la liste complète des utilisateurs avec leurs informations principales : identifiant, nom d'utilisateur, email et rôles. Les rôles sont affichés sous forme de badges colorés pour mieux distinguer les comptes administrateurs (ROLE_ADMIN) des comptes classiques (ROLE_USER)

Quand un administrateur souhaite modifier ou créer un utilisateur, un formulaire modal est affiché (Annexe 18)

Une fois le formulaire modifié, les données sont envoyées à l'API :

```javascript
// Envoyer la requête
fetch(`/api/gamelab/users/${userId}`, {
    method: 'PUT',
    headers: {
        'Content-Type': 'application/json'
    },
    body: JSON.stringify(data)
})
```

Une requête HTTP de type PUT est utilisée ici pour mettre à jour les données côté serveur. Si un mot de passe est saisi, il est ajouté au payload sous la clé plainPassword. Une fois la mise à jour réussie, la liste des utilisateurs est rechargée dynamiquement

C'est le même système pour créer un utilisateur sauf que la requête fetch est ''/api/gamelab/users"

Quand une requête PUT, POST ou DELETE est envoyée depuis le front-end via JavaScript (comme vu précédemment avec fetch()), c'est le contrôleur Symfony AdminUserController qui la réceptionne.

```php
#[Route('/{id}', name: 'app_admin_users_update', methods: ['PUT'])]
public function update(Request $request, int $id): JsonResponse
{
    $this->checkAdminAccess();

    $data = json_decode($request->getContent(), true);
    $response = $this->gamelabApiService->makeApiRequest('PUT', "/api/users/{$id}", $data);

    return $this->json($response);
}
```

7.3-Système d'ajout dans une collection

Le système d'ajout d'un jeu dans une collection se fait via des boutons interactifs présents sur la page de détails du jeu. Ces boutons permettent à l'utilisateur connecté de classer un jeu dans différentes collections personnalisées (comme "Favoris", "Backlog", "Terminé", etc.) ou d'en créer de nouvelles.

La mise dans une collection se fait via le javascript du bouton, c'est le javascript qui gère les interactions du back-end via la fonction fetch.

```
fetch(statusUrl, {
  credentials: 'include',
  headers: { Accept: 'application/json' }
})
  .then(r => {
    if (!r.ok) throw new Error(`HTTP ${r.status}`);
    return r.json();
  })
  .then(j => {
    isFav = j.isFavorite;
    updateUI();
  })
  .catch(error => {
    console.error('Erreur lors de la récupération du statut de favori:', error);
    // On garde l'état initial du bouton en cas d'erreur
  });
```

En utilisant l'AJAX (Asynchronous JavaScript and XML), les actions comme l'ajout ou le retrait d'un jeu dans une collection peuvent être effectuées sans recharger la page. Cela présente plusieurs avantages :

- Meilleure UX : l'utilisateur voit le changement instantanément (icône mise à jour, message de confirmation).
- Réduction des temps de chargement : seules les données nécessaires sont transmises, évitant de renvoyer tout le HTML.
- Feedback immédiat : on peut afficher des messages ou animations (toasts, effets visuels) qui renforcent l'interaction.
- Flexibilité : les boutons peuvent être facilement adaptés pour d'autres collections (Backlog, Wishlist, etc.) sans modifier la structure HTML de la page.

Pour que cette communication AJAX fonctionne, j'ai développé un FavoriteController qui expose des routes dédiées au traitement de ces actions. Ces routes reçoivent les requêtes envoyées via fetch() en JavaScript, exécutent la logique métier (ajout, suppression, vérification de statut), puis renvoient une réponse au format JSON.

```php
#[Route('/game/{gameId}/favorite', name: 'favorite_toggle', methods: ['POST','DELETE'])]
public function toggle(int $gameId, Request $request, GamelabApiService $apiService): JsonResponse
{
    $sessionUser = $request->getSession()->get('user');
    if (!$sessionUser) {
        return $this->json(['message'=>'Non autorisé'], Response::HTTP_UNAUTHORIZED);
    }

    $add    = $request->isMethod('POST');
    $result = $apiService->toggleFavorite($gameId, $add);

    if (!$result['success']) {
        return $this->json(
            ['message' => $result['error'] ?? 'Erreur lors de l'opération'],
            Response::HTTP_BAD_REQUEST
        );
    }

    return $this->json(
        ['isFavorite' => $add],
        $add ? Response::HTTP_CREATED : Response::HTTP_OK
    );
}
```

8-Application Mobile

8.1-Pourquoi avoir choisi une application mobile séparée ?

J'ai choisi de faire une application mobile séparée parce que je trouvais que ça pouvait vraiment servir dans des situations concrètes. Par exemple, si quelqu'un est en brocante ou dans un magasin de jeux vidéo, il peut avoir besoin de consulter rapidement sa collection pour éviter d'acheter un jeu qu'il a déjà. Dans ce cas-là, c'est beaucoup plus simple d'avoir une application dédiée sur son téléphone plutôt que d'aller chercher le site web, se connecter, naviguer, etc.

Je trouvais ça plus pratique, plus rapide, et surtout plus naturel à utiliser dans ce genre de contexte mobile.

Au-delà de ça, il y a d'autres avantages à proposer une app mobile séparée :

- Accès instantané via une icône sur l'écran d'accueil, sans passer par un navigateur.
- Navigation plus fluide, avec une interface pensée pour le tactile et les petits écrans.
- Utilisable en situation de mobilité, même avec une connexion lente ou instable.
- Consultation rapide de sa collection, ce qui est utile pour les collectionneurs ou passionnés.
- Séparation des usages : le site web est parfait pour gérer, noter, commenter, alors que l'app mobile est idéale pour vérifier, consulter, agir vite.

De plus, c'était aussi un bon défi technique de voir comment adapter un projet web pour le mobile dans un langage différent.

8.2-Récupération des différentes API et utilisation

8.2.1 Services de l'api Gamelab

Pour que l'application mobile puisse fonctionner comme la version web, j'ai dû connecter l'app à plusieurs API : L'API Gamelab que j'ai séparé en plusieurs services et Providers différents et le service de l'API IGDB.

Par exemple pour la connexion, plutôt que d'utiliser un système comme Firebase, j'ai préféré faire mon propre système d'auth, connecté à mon backend Symfony. Je trouvais ça plus logique pour rester cohérent avec mon API, gérer les rôles, les tokens, et garder la main sur tout.

J'ai créé un service AuthService en Dart, qui s'occupe de faire les requêtes HTTP pour s'inscrire, se connecter, se déconnecter, etc. Par exemple, la fonction de connexion envoie une requête POST à /api/login :

```dart
Future<User> login(String email, String password) async {
  try {
    final url = '$baseUrl/login';
    debugPrint('Tentative de connexion avec email: $email');
    debugPrint('URL: $url');

    final body = jsonEncode({
      'email': email,
      'password': password,
    });
    debugPrint('Body: $body');

    final response = await http.post(
      Uri.parse(url),
      headers: _getHeaders(),
      body: body,
    ).timeout(timeout);
```

Si la réponse est bonne, je sauvegarde localement le token, le mail, le pseudo et les rôles avec SharedPreferences :

```dart
    // Sauvegarder les informations de l'utilisateur
    await prefs.setString('email', email);
    await prefs.setString('username', data['username'] ?? email.split('@')[0]);
    if (data['roles'] != null) {
      await prefs.setStringList('roles', List<String>.from(data['roles']));
    } else {
      await prefs.setStringList('roles', ['ROLE_USER']);
    }
```

Shared Preferences étant un plugin Flutter permettant de stocker des informations localement sous forme de paires clé-valeur. C'est très utile pour mémoriser les sessions de l'utilisateur entre deux ouvertures de l'application.

Je l'ai installé en suivant la documentation officielle trouvable sur le site pub.dev là ou on peut trouver tout de sorte de plugins flutter :

https://pub.dev/packages/shared_preferences/install

```
$ flutter pub add shared_preferences
```

Puis je crée un objet User et je le renvoie :

```dart
final user = User(
  id: data['id']?.toString() ?? '0',
  username: data['username'] ?? email.split('@')[0],
  email: email,
  roles: data['roles'] != null ? List<String>.from(data['roles']) : ['ROLE_USER'],
); // User
```

Enfin, j'utilise un StreamController pour notifier toute l'app qu'un utilisateur s'est connecté ou déconnecté :

```dart
// Stream pour notifier les changements d'état d'authentification
final StreamController<User?> _authStateController = StreamController<User?>.broadcast();
Stream<User?> get authStateChanges => _authStateController.stream;
```

Pour que l'état de connexion soit accessible partout dans l'application, j'ai utilisé un AuthProvider avec le package provider de Flutter. Ce provider écoute les changements d'état venant du AuthService, et permet d'afficher dynamiquement les bons écrans ou composants selon que l'utilisateur est connecté, administrateur, ou en cours de chargement.

Dès l'instanciation, AuthProvider appelle _initializeAuth() pour savoir si un utilisateur est déjà connecté grâce aux données locales (SharedPreferences) :

```dart
AuthProvider() {
  _initializeAuth();
}

User? get currentUser => _currentUser;
bool get isAuthenticated => _currentUser != null;
bool get isLoading => _isLoading;
String? get error => _error;
bool get isAdmin => _currentUser?.isAdmin ?? false;

Future<void> _initializeAuth() async {
  _isLoading = true;
  notifyListeners();
```

La méthode commence par activer un flag isLoading (utile pour afficher un loader dans l'UI), puis elle appelle :

```
Future<void> _initializeAuth() async {
  _isLoading = true;
  notifyListeners();

  try {
    _currentUser = await _authService.getCurrentUser();
  } catch (e) {
    _error = e.toString();
  } finally {
    _isLoading = false;
    notifyListeners();
  }
```

Et ensuite, elle écoute le stream du StreamController :

```
// Écouter les changements d'état d'authentification
_authService.authStateChanges.listen((user) {
  _currentUser = user;
  notifyListeners();
});
```

Cela veut dire que dès qu'un changement d'état de l'utilisateur se produit (connexion, déconnexion), l'interface est mise à jour automatiquement.

J'ai choisi d'utiliser un AuthProvider pour structurer proprement toute la logique d'authentification dans l'application. Cela me permet de centraliser l'état de l'utilisateur (connecté, administrateur, en cours de chargement, etc.) et d'y accéder facilement depuis n'importe quel écran. C'est aussi une approche plus sécurisée, car les données sensibles comme le token ou les rôles ne sont jamais manipulées directement dans les widgets, mais uniquement via le provider

8.2.2 Services de l'api IGDB

Pour le Service de l'api IGDB j'ai réalisé de la même manière que pour l'application web, le service centralise les endpoints et requêtes de l'api pour pouvoir les réutiliser dans l'ensemble de mon projet.

Par exemple, pour la méthode getAllGenres, j'envoie une requête POST à l'endpoint /genres de l'API IGDB avec une query spécifique qui demande les champs id et name, triés par

nom. Je passe les headers nécessaires à l'authentification (Client-ID et Bearer Token), puis je traite la réponse : si le statut est 200, je décode le JSON et je le transforme en liste d'objets Genre. Sinon, je lance une exception avec le code d'erreur. Cette approche rend le code propre, modulaire et facilement maintenable.

```dart
Future<List<Genre>> getAllGenres(String query) async {
  final String _genresUrl = '$_baseUrl/genres';

  final String query = '''
    fields id, name;
    limit 500;
    sort name asc;
  ''';

  final response = await http.post(
    Uri.parse(_genresUrl),
    headers: {
      'Client-ID': _clientID,
      'Authorization': 'Bearer $_accessToken',
      'Content-Type': 'application/json',
    },
    body: query,
  );

  if (response.statusCode == 200) {
    List<dynamic> data = jsonDecode(response.body);
    return data.map((json) => Genre.fromJson(json)).toList();
  } else {
    throw Exception(
        'Failed to load games genres. Status code: ${response.statusCode}');
  }
}
```

8.3-Création de l'APK et installation de cette dernière

Pour générer une APK avec Flutter, j'ai d'abord vérifié que mon environnement était bien configuré pour Android en utilisant la commande flutter doctor. Une fois tout en ordre, j'ai utilisé la commande suivante dans le terminal à la racine de mon projet :

flutter build apk --release

Pour installer cette APK sur un smartphone Android via USB, j'ai d'abord activé les options développeur sur le téléphone ainsi que le débogage USB. Ensuite, j'ai connecté le téléphone au PC avec un câble USB. Une fois reconnu par j'ai transféré sur le dossier téléchargement de mon smartphone et j'ai installé l'application.

9-Futures Améliorations du projet

Il est important pour moi de définir dans ce dossier les améliorations futures que je pourrais apporter à Gamelab. Au-delà des contraintes évoquées à la page 6, j'ai identifié plusieurs pistes d'amélioration pertinentes à envisager par la suite. Il s'agit de fonctionnalités secondaires que je n'ai pas pu intégrer durant la phase de développement actuelle, mais auxquelles j'ai réfléchi en vue d'une évolution future du projet.

Titre	Description
Mode Hors Ligne	Un réel mode hors ligne où on a accès soit à toute la librairie offerte par IGDB soit à nos jeux mis dans des collections, elle ne sera accessible que sur Téléphone, ce qui donne un réel plus à l'application mobile.
Multilingue	Avoir Accès à plusiuers langues / toutes les langues du monde, traduit automatiquement par ia.
Intégration API Twtich	Pour pouvoir voir quel Streamer est en live sur le jeu que l'on souhaite
Intégration de HowLongToBeat	HowLongToBeat est un site qui répertori le nombre d'heure qu'il faut pour finir un jeu ou un DLC, il ne possède pas d'API publique mais s'associer avec ce site serait une très belle idée
Prix des jeux	Avoir un visuel ou accès au prix des jeux en 1 clic
Guide des joueurs	Permettre aux joueurs d'élaborer leur propre guide pour finir un jeu, débloquer des succès ou autre, comme sur Steam

10-Conclusion

La réalisation du projet Gamelab a constitué une expérience particulièrement enrichissante sur les plans technique et professionnel. Ce projet m'a permis de mettre en pratique, de manière concrète, l'ensemble des compétences acquises durant ma formation de Concepteur Développeur d'Applications, que ce soit en conception, en développement fullstack ou en gestion de projet.

J'ai eu l'opportunité de développer une application complète et multiplateforme, en intégrant notamment la création d'une API en Symfony, ainsi que la consommation d'une API tierce, me confrontant ainsi à des problématiques réelles de production, de structuration du code et de performances.

Le rythme de l'alternance s'est révélé particulièrement formateur. Travailler en entreprise tout en suivant une formation m'a permis de renforcer mes acquis, d'apprendre à m'adapter rapidement, et surtout de gagner en autonomie, en rigueur et en professionnalisme.

Fort de cette expérience, je souhaite désormais poursuivre mon évolution en intégrant le cursus Architecte Web (Bac +5), au sein de ForEach Academy, tout en poursuivant mon alternance chez Almadia, une entreprise dans laquelle je m'épanouis.

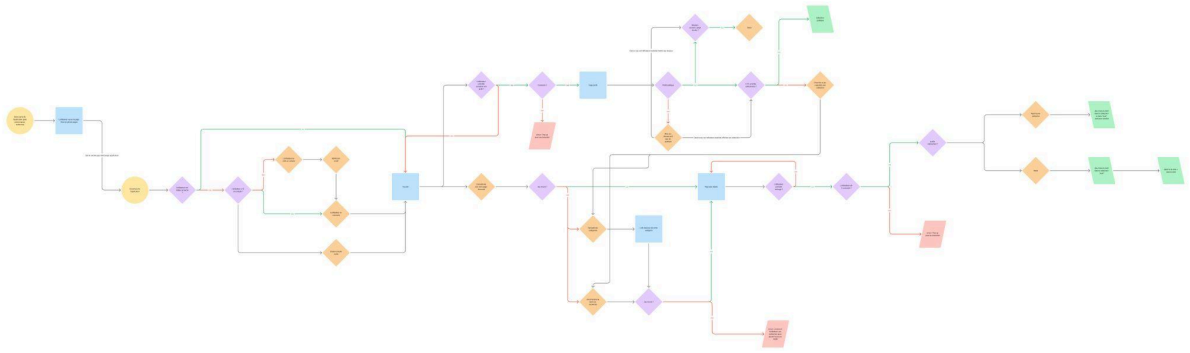
Concernant Gamelab, si l'occasion se présente, j'aimerais sincèrement poursuivre son développement. Ce projet me tient à cœur, et je vois en lui un fort potentiel : non seulement pour approfondir mes compétences techniques, mais aussi pour créer une véritable communauté autour du jeu. Mon ambition serait de faire évoluer Gamelab afin d'offrir une expérience toujours plus riche et engageante à plusieurs milliers de joueurs.

Merci de m'avoir accordé votre attention à la lecture de ce dossier. J'espère qu'il aura su refléter l'implication, les compétences et la passion que j'ai investies dans ce projet.

Matthieu Nowak.

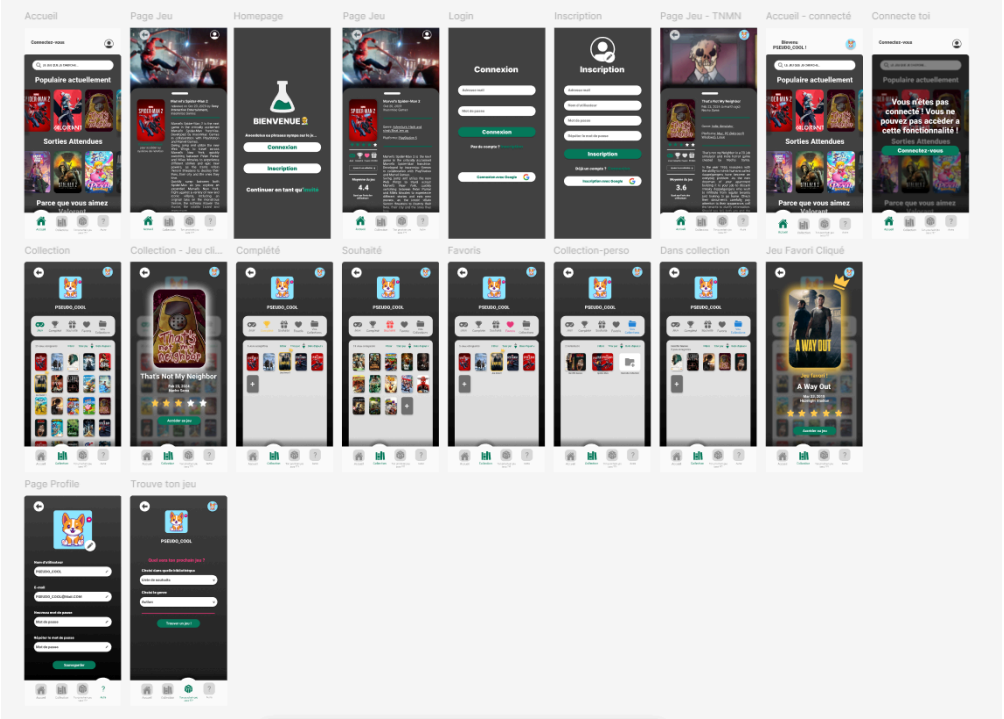
11-Annexes

Annexe 1 : Parcours utilisateur

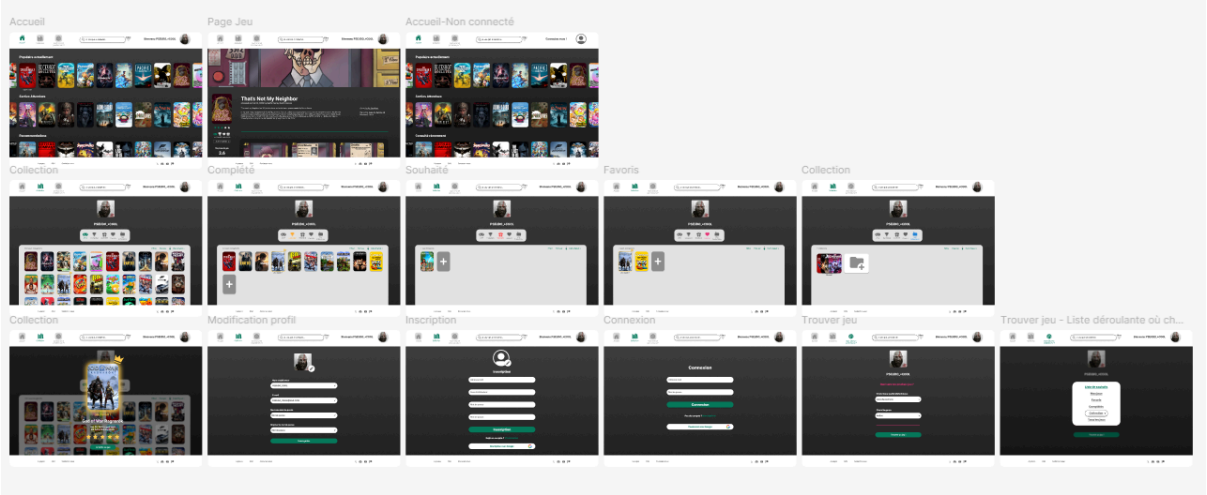


https://www.figma.com/board/Yq9C8v963lUbBty8hxMJVf/Diagramme-Userflow---Matthieu-NOWAK---Gamelab?node-id=0-1&t=ZcA8lyl16x2XCi1i-1

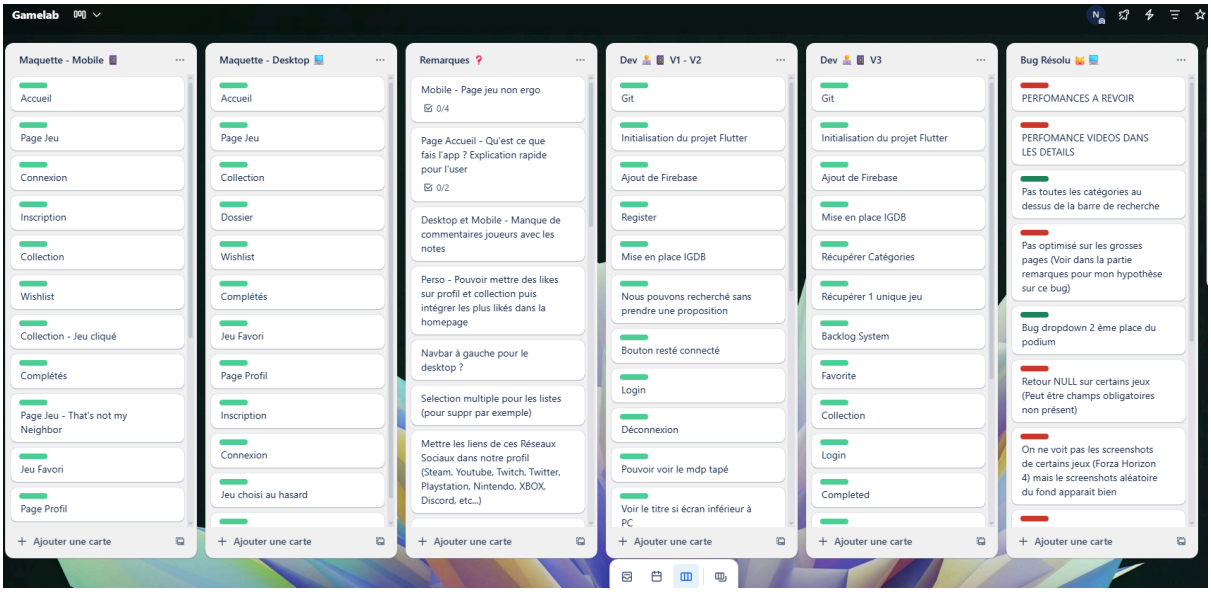
Annexe 2 : Maquette Mobile



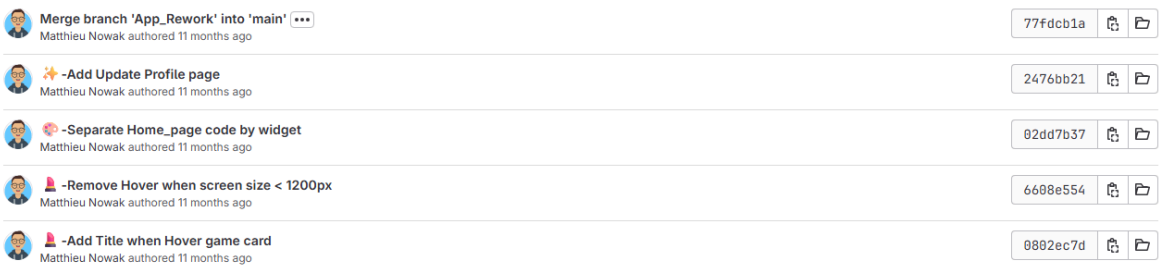
Annexe 3 : Maquette Web



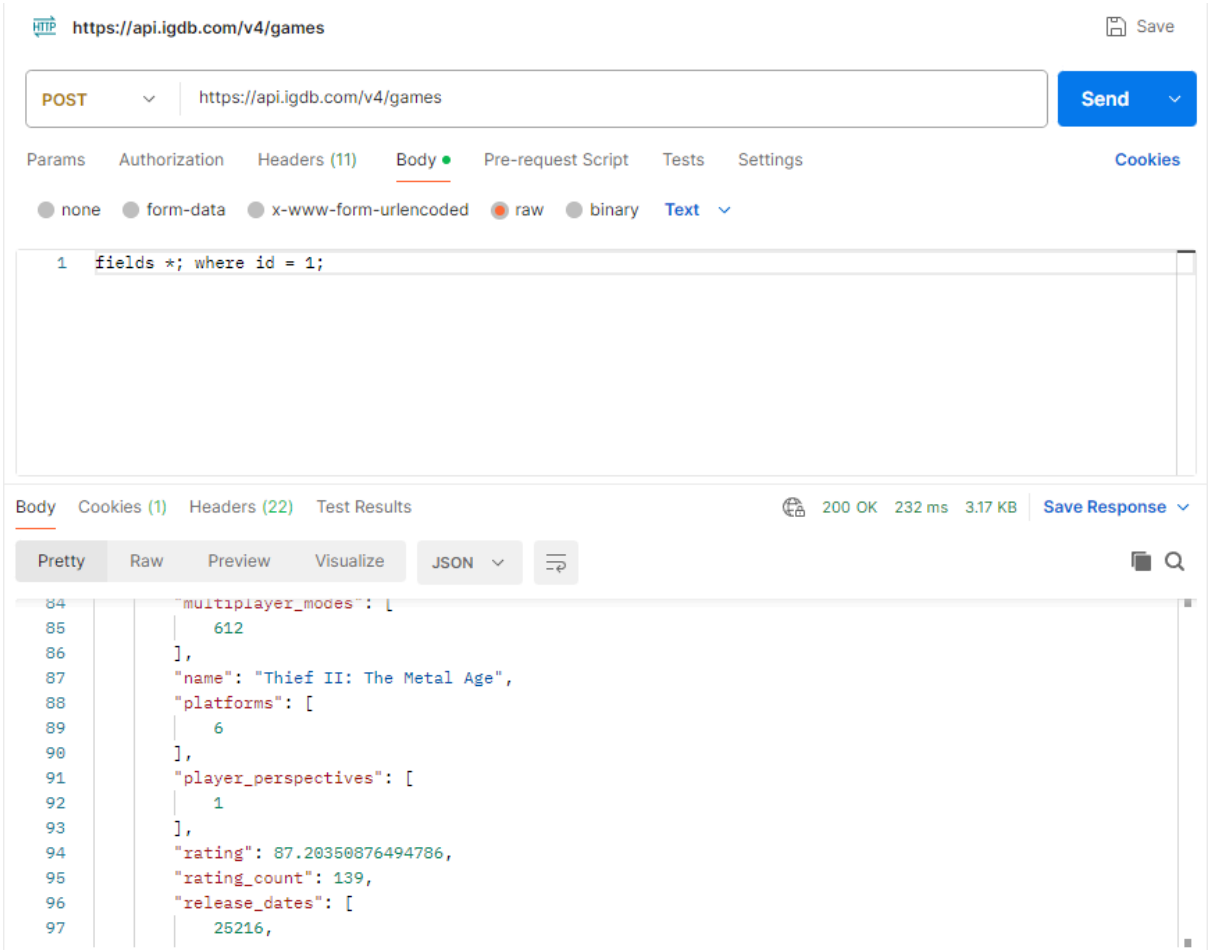
Annexe 4 : Tableau Trello Gamelab



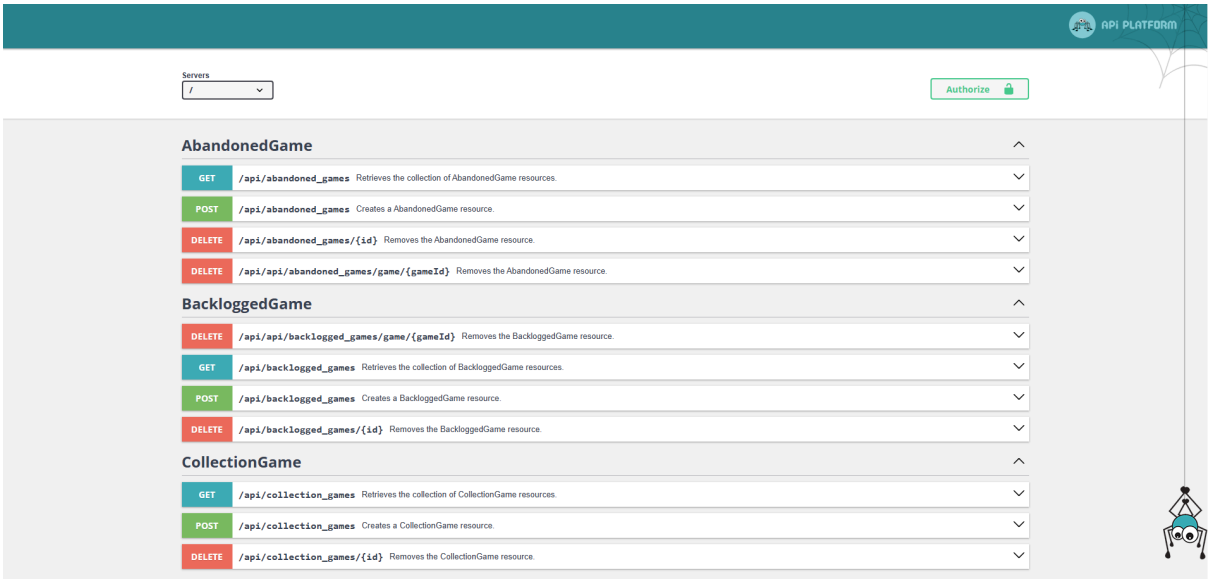
Annexe 5 : Gitlab avec utilisation de GitMoji



Annexe 6 : Test Postman pour récupérer un jeu via un ID sur l'API IGDB



Annexe 7 : Visuel de API Platform :



Annexe 8 : Exemple fichier de migration via doctrine

```php
<?php

declare(strict_types=1);

namespace DoctrineMigrations;

use Doctrine\DBAL\Schema\Schema;
use Doctrine\Migrations\AbstractMigration;

/**
 * Auto-generated Migration: Please modify to your needs!
 */
final class Version20250516153030 extends AbstractMigration
{
    public function getDescription(): string
    {
        return '';
    }

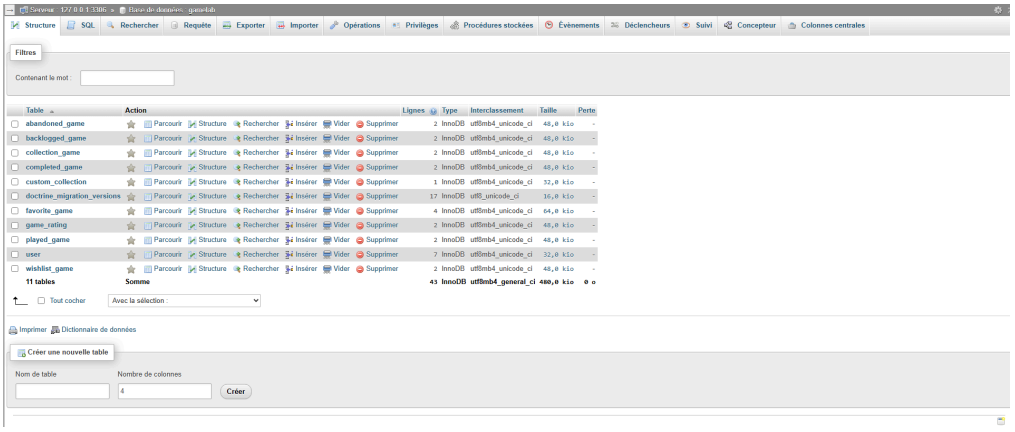
    public function up(Schema $schema): void
    {
        // this up() migration is auto-generated, please modify it to
your needs
        $this->addSql('ALTER TABLE abandoned_game DROP FOREIGN KEY
FK_73E325A6A76ED395');
        $this->addSql('DROP INDEX uniq_user_abandoned_game ON
abandoned_game');
        $this->addSql('CREATE UNIQUE INDEX UNIQ_USER_GAME_ABANDONED ON
abandoned_game (user_id, game_id)');
        $this->addSql('ALTER TABLE abandoned_game ADD CONSTRAINT
FK_73E325A6A76ED395 FOREIGN KEY (user_id) REFERENCES user (id)');
        $this->addSql('ALTER TABLE backlogged_game DROP FOREIGN KEY
FK_AC896115A76ED395');
        $this->addSql('DROP INDEX uniq_user_backlogged_game ON
backlogged_game');
        $this->addSql('CREATE UNIQUE INDEX UNIQ_USER_GAME_BACKLOGGED ON
backlogged_game (user_id, game_id)');
        $this->addSql('ALTER TABLE backlogged_game ADD CONSTRAINT
FK_AC896115A76ED395 FOREIGN KEY (user_id) REFERENCES user (id)');
        $this->addSql('ALTER TABLE completed_game DROP FOREIGN KEY
FK_47A9C4B0A76ED395');
```

```
        $this->addSql('DROP INDEX uniq_user_completed_game ON
completed_game');
        $this->addSql('CREATE UNIQUE INDEX UNIQ_USER_GAME_COMPLETED ON
completed_game (user_id, game_id)');
        $this->addSql('ALTER TABLE completed_game ADD CONSTRAINT
FK_47A9C4B0A76ED395 FOREIGN KEY (user_id) REFERENCES user (id)');
        $this->addSql('ALTER TABLE played_game DROP FOREIGN KEY
FK_54BE8039A76ED395');
        $this->addSql('DROP INDEX uniq_user_played_game ON
played_game');
        $this->addSql('CREATE UNIQUE INDEX UNIQ_USER_GAME_PLAYED ON
played_game (user_id, game_id)');
        $this->addSql('ALTER TABLE played_game ADD CONSTRAINT
FK_54BE8039A76ED395 FOREIGN KEY (user_id) REFERENCES user (id)');
        $this->addSql('ALTER TABLE wishlist_game DROP FOREIGN KEY
FK_5814E07AA76ED395');
        $this->addSql('DROP INDEX uniq_user_wishlist_game ON
wishlist_game');
        $this->addSql('CREATE UNIQUE INDEX UNIQ_USER_GAME_WISHLIST ON
wishlist_game (user_id, game_id)');
        $this->addSql('ALTER TABLE wishlist_game ADD CONSTRAINT
FK_5814E07AA76ED395 FOREIGN KEY (user_id) REFERENCES user (id)');
    }

    public function down(Schema $schema): void
    {
        // this down() migration is auto-generated, please modify it to
your needs
        $this->addSql('ALTER TABLE abandoned_game DROP FOREIGN KEY
FK_73E325A6A76ED395');
        $this->addSql('DROP INDEX uniq_user_game_abandoned ON
abandoned_game');
        $this->addSql('CREATE UNIQUE INDEX UNIQ_USER_ABANDONED_GAME ON
abandoned_game (user_id, game_id)');
        $this->addSql('ALTER TABLE abandoned_game ADD CONSTRAINT
FK_73E325A6A76ED395 FOREIGN KEY (user_id) REFERENCES user (id)');
        $this->addSql('ALTER TABLE backlogged_game DROP FOREIGN KEY
FK_AC896115A76ED395');
        $this->addSql('DROP INDEX uniq_user_game_backlogged ON
backlogged_game');
        $this->addSql('CREATE UNIQUE INDEX UNIQ_USER_BACKLOGGED_GAME ON
backlogged_game (user_id, game_id)');
```

```
        $this->addSql('ALTER TABLE backlogged_game ADD CONSTRAINT
FK_AC896115A76ED395 FOREIGN KEY (user_id) REFERENCES user (id)');
        $this->addSql('ALTER TABLE completed_game DROP FOREIGN KEY
FK_47A9C4B0A76ED395');
        $this->addSql('DROP INDEX uniq_user_game_completed ON
completed_game');
        $this->addSql('CREATE UNIQUE INDEX UNIQ_USER_COMPLETED_GAME ON
completed_game (user_id, game_id)');
        $this->addSql('ALTER TABLE completed_game ADD CONSTRAINT
FK_47A9C4B0A76ED395 FOREIGN KEY (user_id) REFERENCES user (id)');
        $this->addSql('ALTER TABLE played_game DROP FOREIGN KEY
FK_54BE8039A76ED395');
        $this->addSql('DROP INDEX uniq_user_game_played ON
played_game');
        $this->addSql('CREATE UNIQUE INDEX UNIQ_USER_PLAYED_GAME ON
played_game (user_id, game_id)');
        $this->addSql('ALTER TABLE played_game ADD CONSTRAINT
FK_54BE8039A76ED395 FOREIGN KEY (user_id) REFERENCES user (id)');
        $this->addSql('ALTER TABLE wishlist_game DROP FOREIGN KEY
FK_5814E07AA76ED395');
        $this->addSql('DROP INDEX uniq_user_game_wishlist ON
wishlist_game');
        $this->addSql('CREATE UNIQUE INDEX UNIQ_USER_WISHLIST_GAME ON
wishlist_game (user_id, game_id)');
        $this->addSql('ALTER TABLE wishlist_game ADD CONSTRAINT
FK_5814E07AA76ED395 FOREIGN KEY (user_id) REFERENCES user (id)');
    }
}
```

Annexe 9 : Page PhpMyAdmin



Annexe 10 : SecurityController.php pour pouvoir gérer la connexion / déconnexion

```php
class SecurityController extends AbstractController
{
    #[Route (path: '/api/login', name: 'api_login', methods: ['POST'])]
    public function login(Request $request, #[CurrentUser] ?User $user, TokenStorageInterface $tokenStorage){
        if (!$user) {
            return $this->json(['message' => 'Identifiants invalides'], JsonResponse::HTTP_UNAUTHORIZED);
        }

        // Créer un token d'authentification
        $token = new UsernamePasswordToken($user, 'main', $user->getRoles());

        // Stocker le token dans le token storage (établit la session)
        $tokenStorage->setToken($token);

        // Obtenir la session et la marquer comme initialisée
        $session = $request->getSession();
        $session->set('_security_main', serialize($token));
        $session->save();

        // Retourner les informations de l'utilisateur
        return $this->json([
            'email' => $user->getEmail(),
            'roles' => $user->getRoles(),
            'username' => $user->getUsername(),
        ], Response::HTTP_OK, []);
    }

    #[Route (path: '/api/logout', name: 'api_logout', methods: ['POST'])]
    public function logout(TokenStorageInterface $tokenStorage, Request $request){
        // Session invalide
        $session = $request->getSession();
        $session->invalidate();

        // Supprimer le token de sécurité
        $tokenStorage->setToken(null);

        return new JsonResponse(['message' => 'Vous avez été déconnecté'], JsonResponse::HTTP_OK);
    }
}
```

Annexe 11 : Code FavoriteGameController pour gérer les jeux favoris d'un utilisateur

```php
#[Route('/api/favorites')]
#[IsGranted('ROLE_USER')]
class FavoriteGameController extends AbstractController
{
    public function __construct(
        private FavoriteGameRepository $favoriteRepo,
        private IgdbService $igdbService,
        private EntityManagerInterface $em
    ) {}

    #[Route('', name: 'api_favorites_add', methods: ['POST'])]
    public function add(Request $request): JsonResponse
    {
        $data = json_decode($request->getContent(), true);
        if (empty($data['gameId']) || !is_int($data['gameId'])) {
```

```
            return $this->json(['error'=>'L\'ID du jeu est requis'],
Response::HTTP_BAD_REQUEST);
        }
        $gameId = $data['gameId'];
        $user   = $this->getUser();

        if ($this->favoriteRepo->findOneByUserAndGameId($user,
$gameId)) {
            return $this->json(['error'=>'Déjà en favoris'],
Response::HTTP_BAD_REQUEST);
        }

        try {
            $gameName =
$this->igdbService->fetchGameById($gameId)->getName();
        } catch (\Exception $e) {
            return $this->json(['error'=>'IGDB error:
'.$e->getMessage()], Response::HTTP_BAD_REQUEST);
        }

        $fav = (new FavoriteGame())
            ->setUser($user)
            ->setGameId($gameId)
            ->setGameName($gameName);

        $this->em->persist($fav);
        $this->em->flush();

        return $this->json([
            'id'       => $fav->getId(),
            'gameId'   => $fav->getGameId(),
            'gameName' => $fav->getGameName(),
        ], Response::HTTP_CREATED);
    }

    #[Route('', name: 'api_favorites_list', methods: ['GET'])]
    public function list(): JsonResponse
    {
        $user = $this->getUser();
        $favorites = $this->favoriteRepo->findBy(['user' => $user]);

        $data = [];
        foreach ($favorites as $favorite) {
```

```
            $data[] = [
                'id' => $favorite->getId(),
                'gameId' => $favorite->getGameId(),
                'gameName' => $favorite->getGameName(),
            ];
        }

        return $this->json($data);
    }

    #[Route('/{gameId}', name: 'api_favorites_remove', methods:
['DELETE'])]
    public function remove(int $gameId): JsonResponse
    {
        $user = $this->getUser();
        $favorite = $this->favoriteRepo->findOneByUserAndGameId($user,
$gameId);

        if (!$favorite) {
            return $this->json(['error' => 'Favori non trouvé'],
Response::HTTP_NOT_FOUND);
        }

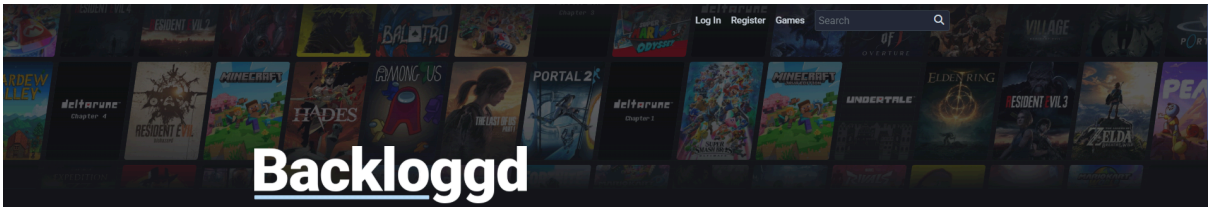
        $this->em->remove($favorite);
        $this->em->flush();

        return $this->json(['message' => 'Favori supprimé']);
    }

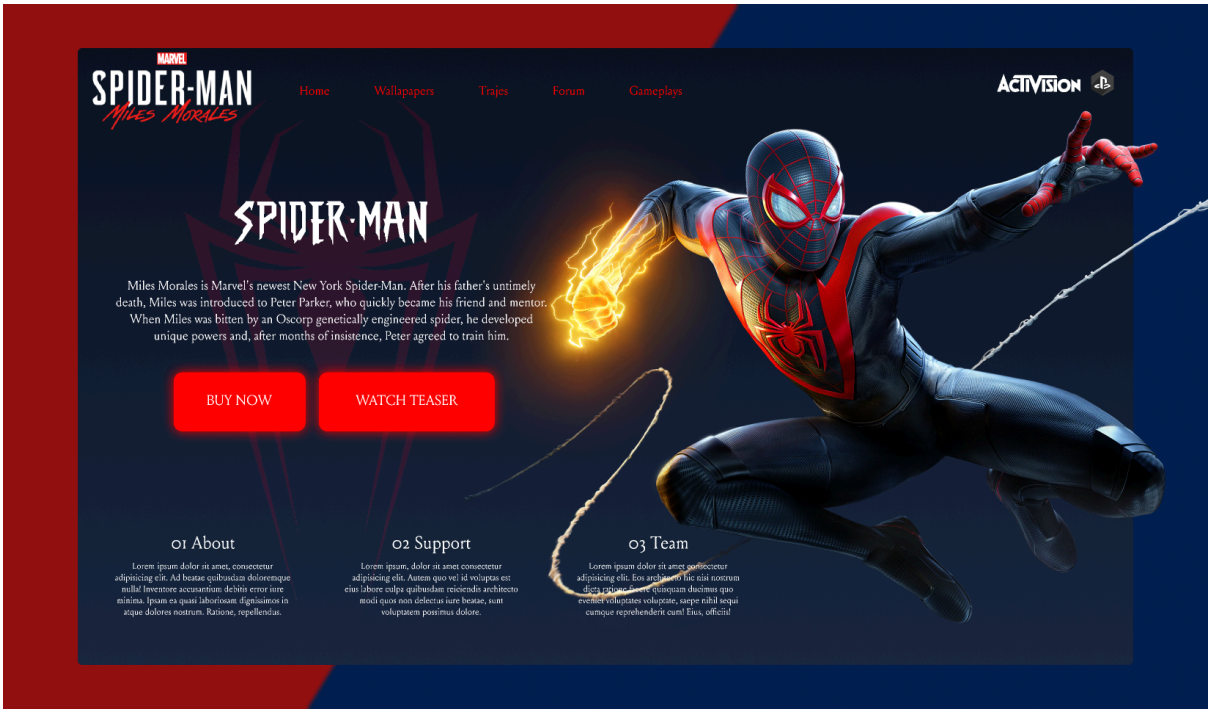
    #[Route('/status/{gameId}', name: 'api_favorites_status', methods:
['GET'])]
    public function status(int $gameId): JsonResponse
    {
        $user = $this->getUser();
        $isFavorite =
$this->favoriteRepo->findOneByUserAndGameId($user, $gameId) !== null;

        return $this->json(['isFavorite' => $isFavorite]);
    }
}
```

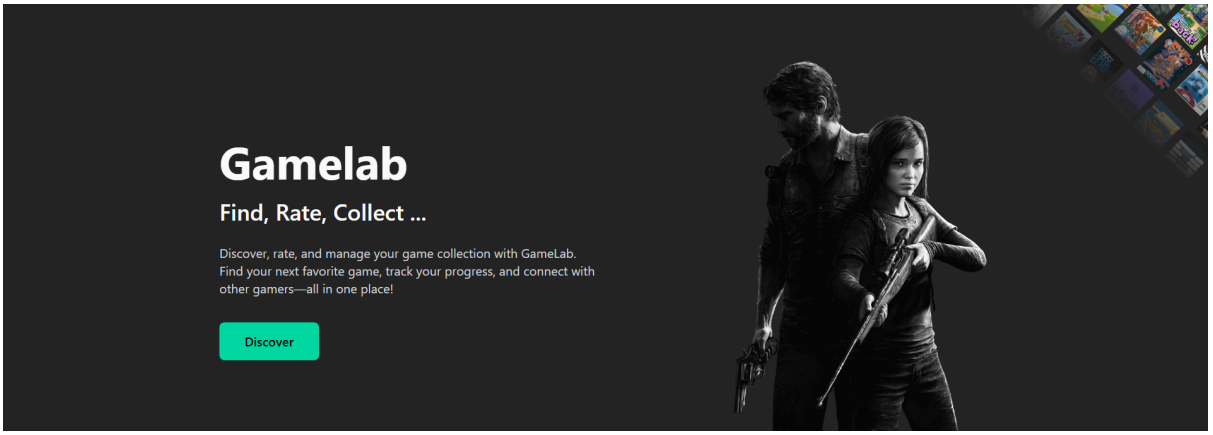
Annexe 12 : Inspiration Maquette Backloggd

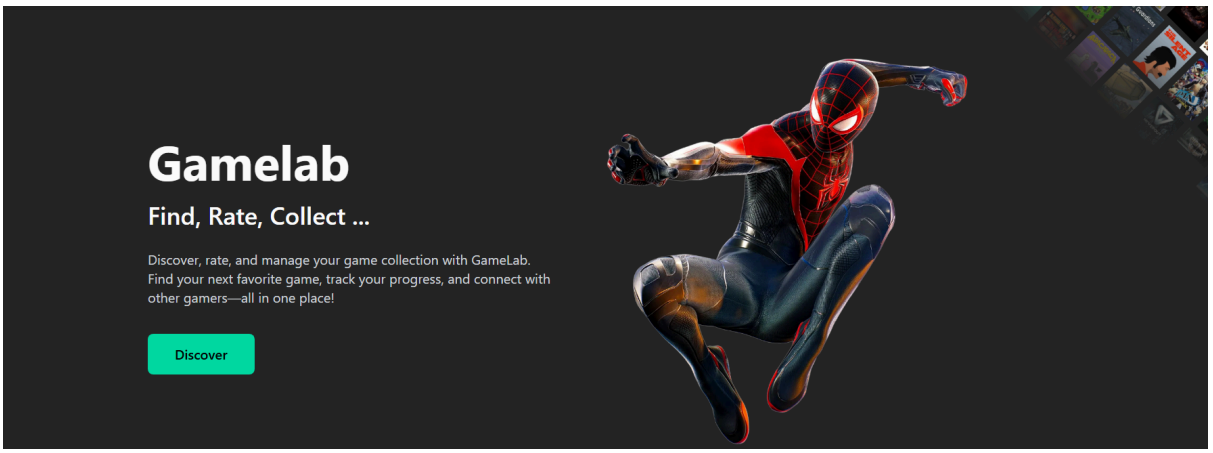


Annexe 13 : Inspiration Maquette Miles Morales



Annexe 14 : Section Jaquette et personnage aléatoire Landing Page Gamelab





Annexe 15 : HomeController de la landing page qui gère l'aléatoire :

```php
<?php

namespace App\Controller;

use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;
use Symfony\Contracts\HttpClient\HttpClientInterface;
use Symfony\Contracts\Cache\CacheInterface;

final class HomeController extends AbstractController
{
    public function __construct(
        private HttpClientInterface $client,
        private CacheInterface $cache
    ) {}

    #[Route('/', name: 'app_home')]
    public function index(): Response
    {
        // Sélection aléatoire d'un personnage
        $characters = [
            '/images/characters/Joel-Ellie.webp',
            '/images/characters/Miles-Morales.webp',
            '/images/characters/Spider-man.webp',
            '/images/characters/Master-chief.webp',
            '/images/characters/Jinx.webp',
        ];
        $randomCharacter = $characters[array_rand($characters)];
```

```
        // On récupère 3 listes de jeux aléatoires
        $gamesLine1 = $this->fetchRandomGames(12);
        $gamesLine2 = $this->fetchRandomGames(12);
        $gamesLine3 = $this->fetchRandomGames(12);

        return $this->render('pages/home_page/home_page.html.twig', [
            'character'  => $randomCharacter,
            'gamesLine1' => $gamesLine1,
            'gamesLine2' => $gamesLine2,
            'gamesLine3' => $gamesLine3,
        ]);
    }

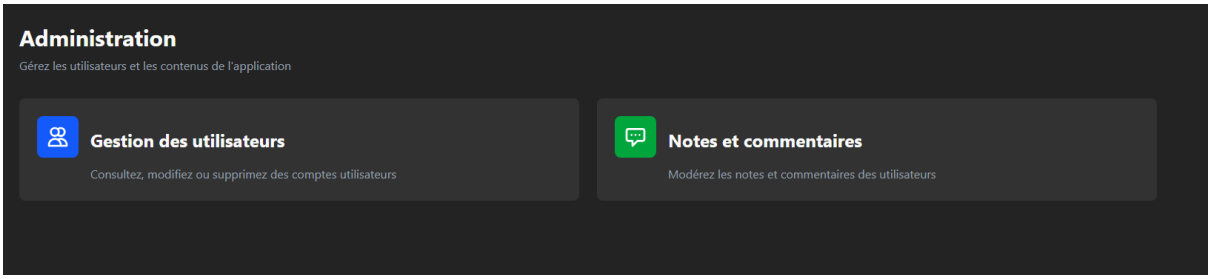
    /**
     * Récupère un nombre donné de jeux aléatoires sur IGDB
     */
    private function fetchRandomGames(int $limit): array
    {
        $offset = random_int(0, 50_000);

        // On ne récupère que les jeux qui ont une cover
        $query = sprintf(
            'fields name, cover.url, summary;
             where cover.url != null;
             limit %d;
             offset %d;',
            $limit,
            $offset
        );

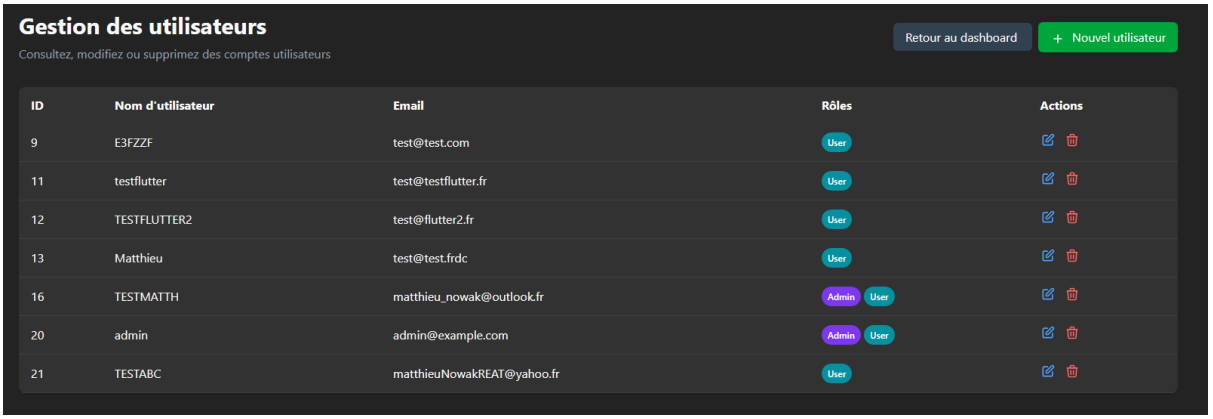
        $response = $this->client->request('POST',
'https://api.igdb.com/v4/games', [
            'headers' => [
                'Client-ID'     => 'MONIDCLIENT',
                'Authorization' => 'Bearer MONBEARERID',
            ],
            'body' => $query,
        ]);

        return $response->toArray();
    }
}
```

Annexe 16 : Dashboard Administration



Annexe 17 : CRUD Utilisateur Administration



Annexe 18 : Modals quand on souhaite créer et modifier un utilisateur

